WAQAR COMPUTER INSTITUTE

Native Android Application Architecture Blueprint

Kotlin + Jetpack Compose | Firebase / Google Cloud | GitHub Actions CI/CD

450 defined functions - 4 role portals - 100,000-student data model
50 libraries - 50 security controls - server-side AI and live-class integrations

Version 1.0 | 10 November 2026 | Status: proposed architecture for approval
Prepared for: Waqar (Owner / Administrator)

Document Control

Item	Detail
Document	Native Android Architecture Blueprint - Waqar Computer Institute
Version / date	1.0 / 10 November 2026
Audience	Owner, Android developers, backend developer, QA, DevOps
Companion material	13-slide architecture walkthrough video (frames reproduced in Appendix D); 14 reference diagrams (Chapters 4-16)
Product type	Native Android app written in Kotlin with Jetpack Compose. No WebView, no PWA, no web-app files of any kind.
Backend	Firebase (Authentication, Firestore, Storage, Cloud Functions, Messaging, App Check, Crashlytics) on Google Cloud
Build system	Gradle (Android Gradle Plugin) executed by GitHub Actions; signed APK and AAB artifacts
Data policy	Production holds only real data entered by the administrator. No dummy records, no placeholder functions, no mock screens.

How to read this document. It is a design and specification document. It defines what will be built, where each function lives in the admin portal, how data and secrets flow, and how the APK is produced. Writing the Kotlin source, deploying Firebase and testing on devices are separate delivery phases (Chapter 18). Every statement here is a proposal until verified in the real repository and a real Firebase project.

Contents

Document Control 2
Contents 3
1. Purpose, Scope and Non-Negotiables 4
2. Executive Summary 5
3. Requirements Traceability 6
4. System Architecture Overview 7
5. Android Application Architecture 9
6. Authentication, Roles and the Administrator Account 12
7. Firebase and Google Cloud Backend 15
8. Firestore Data Model for 100,000 Students 18
9. Student Save Flow, Realtime Sync and Offline Behaviour 21
10. Admin Portal Information Architecture 23
11. Teacher, Student and Parent Portals 25
12. UI / UX Design System: Glass, Liquid and 3D 26
13. AI, Live Classes and Secret Management 28
14. Security Architecture 31
15. Libraries and Code Organisation 34
16. CI/CD: GitHub Actions to a Signed Native APK 36
17. Testing, Quality and the "No Fake Data" Policy 40
18. Delivery Roadmap 41
19. Risks, Open Decisions and Final Verdict 42
Appendix A. The 450 Functions 44
Appendix B. The 50 Libraries 56
Appendix C. The 50 Security Controls 58
Appendix D. Video Walkthrough Index 60

1. Purpose, Scope and Non-Negotiables

This blueprint describes a complete, production-grade management application for Waqar Computer Institute. It covers the admission-to-certificate lifecycle of a student, staff and payroll, fees and accounts, attendance, examinations, live classes, AI tutoring, notifications, reporting, security and backup. The application is a **native Android app** and is the only client. The administrator, teachers, students and parents each sign in to the same app and see a portal matching their role.

1.1 What this document delivers

- A layered architecture (UI, ViewModel, Use Case, Repository, Data Source, Cloud) with the Kotlin module and file structure.
- A role model and a login design in which the administrator username and password are managed by Firebase Authentication and changeable from inside the app.
- A Firestore data model sized for 100,000 student records, including indexes, paging, bulk import and offline behaviour.
- **450 uniquely numbered functions** (Appendix A), each assigned to a module, a place in the side menu or bottom tab bar, a type and the roles that may use it.
- A design system for glass, liquid and 3D interfaces with performance budgets and fallbacks for low-end phones.
- A server-side design for Gemini, OpenAI and Agora so that no provider secret exists in the APK or the repository.
- **50 curated libraries** (Appendix B) and **50 security controls** (Appendix C).
- Complete GitHub Actions workflows that build, test and sign the native APK/AAB, plus the secrets they need.

1.2 Non-negotiable rules from the brief

#	Rule	How the architecture enforces it
R1	Native Kotlin Android app only; no web app files	Single Gradle Android project with Compose UI. No WebView dependency, no HTML bundle, no Capacitor/Cordova/Flutter. CI fails the build if a WebView import or an assets/www folder appears (Chapter 16).
R2	Fully working, no fake functions	Every function in Appendix A has an owner screen, a use case and a backend path. A function is not "done" until it passes its acceptance test against the real project.
R3	No dummy or demo data	Production Firestore starts empty except the administrator profile created by the bootstrap script. Screens show real empty states, never sample rows.
R4	Admin credentials stored in cloud and changeable in-app	Credentials live in Firebase Authentication (the secure store). Profile and username mapping live in Firestore. Changes made in the app take effect in the cloud immediately (Chapter 6).
R5	100,000 student records	Collection design, composite indexes, cursor paging and bulk-import jobs sized for 100k+ documents (Chapter 8).
R6	Gemini, OpenAI and Agora keys never leak	Keys exist only in Google Secret Manager and are used by Cloud Functions. The APK holds no provider secret (Chapter 13).
R7	APK built by GitHub Actions	Workflows in Chapter 16 produce a signed APK and AAB from a protected keystore.
R8	World-class UI: glass, liquid, 3D	Design-system chapter defines components, libraries, motion rules and graceful fallbacks (Chapter 12).

Honest engineering notes. (1) "Never leak" is achieved by design: a secret that is not in the app cannot be extracted from the app. Any key shipped inside an APK can eventually be recovered, so none is shipped. (2) A password must never be stored as readable text in any database, including Firestore. The administrator password is held by Firebase Authentication as a salted hash, which still satisfies "saved in the cloud and changeable from the app". (3) A fixed price or "Rs 40-50 lakh" look is a quality bar, not a feature; this document treats it as the standard for UI polish, security and reliability.

2. Executive Summary

The proposed system is a native Android application backed by Firebase. The phone talks directly to Firebase Authentication, Firestore, Storage and Messaging through the official SDKs, and every one of those direct calls is governed by security rules and App Check. Anything that needs a secret, a trusted calculation or a third party (Gemini, OpenAI, Agora, payments, bulk jobs, certificates, reports) goes through Cloud Functions that run with their own service accounts.

The Android code follows MVVM with Clean Architecture. Compose screens observe ViewModel state; ViewModels call use cases; use cases call repository interfaces; repository implementations coordinate Firestore, Room cache and Cloud Functions. This keeps business rules testable without a device and lets the data layer change without touching the UI.

2.1 System at a glance

Layer	Technology	Responsibility
Client UI	Jetpack Compose, Material 3, glass / liquid / 3D effects	Screens, navigation, animation, accessibility
Presentation logic	ViewModel + StateFlow	UI state, events, one-shot effects
Domain	Kotlin use cases, entities, repository interfaces	Business rules independent of Android and Firebase
Data	Repository implementations, Firestore SDK, Room, DataStore	Fetch, cache, sync, mapping
Backend	Cloud Functions (TypeScript), Secret Manager, Cloud Scheduler	Trusted logic, provider calls, jobs, webhooks
Platform	Auth, Firestore, Storage, FCM, App Check, Crashlytics	Identity, data, files, push, attestation, telemetry
Delivery	GitHub Actions, Gradle, Firebase App Distribution, Play Console	Test, sign, package, distribute

2.2 Headline numbers

Item	Value
Functions defined	450 across 24 modules (Appendix A)
Role portals	4: Admin, Teacher, Student, Parent
Admin bottom tab bar	5 tabs: Home, Students, Attendance, Fees, Inbox
Admin side menu	7 groups, 17 destinations beyond the tabs
Student capacity	100,000+ records with cursor paging; no screen loads the whole collection
Libraries	50 curated (Appendix B)
Security controls	50 distinct (Appendix C)
Third-party providers	Gemini, OpenAI, Agora - all behind Cloud Functions

3. Requirements Traceability

Each request in the original brief is mapped to the chapter that answers it. Items where the design deliberately differs from the literal wording are flagged so there are no surprises later.

Request in the brief	Design response	Chapter	Note
50-60 page PDF in English	This document	All	-
Architecture in depth	Layers, modules, flows, data model, CI/CD	4-9, 16	-
450 functions defined, with side menu and tab bar placement	Appendix A plus navigation model	10, App. A	Each function carries an ID, owner screen and roles
Very professional UI; glass, liquid, 3D, many libraries	Design system with fallbacks	12	Effects are tiered so low-end phones stay smooth
Database for 100,000 students	Firestore model, indexes, paging, import	8, 9	Offline cache is a subset, not all 100k
Everything the admin saves is stored in the cloud	Write-through repositories, Firestore listeners	9	Offline writes queue and sync when online
Admin username and password (Waqar) in cloud, changeable in-app	Firebase Auth credential + Firestore profile	6	Password never stored as text; username alias mapping
GitHub Actions builds native APK; never a web app	Workflows with guard checks	16	CI blocks WebView and web bundles
50 libraries and 50 security rules like banking apps	Appendices B and C	14, 15	Banking-grade controls adapted to a Firebase stack
Gemini, OpenAI, Agora keys: where and how, never leak	Secret Manager + Cloud Functions proxy	13	Keys never reach the device
No fake functions, no dummy data	Acceptance rules and empty-state policy	1, 17	Load tests use a separate staging project
Attach images and video workflow	14 reference diagrams and 13 video frames	Throughout, App. D	Diagrams are reference material supplied with the brief

4. System Architecture Overview

The walkthrough video opens with the whole system in one picture: four user portals feed one Android app; the app is organised in layers; sensitive work crosses a secure backend; the cloud provides authentication, data, files, functions and monitoring. The figure below is that first slide, and the diagrams that follow refine it.



Figure 4.1 - Whole system at a glance (walkthrough video, slide 1).

The reference diagram supplied with the brief expresses the same idea at component level. Note the split between the Room local database and the Firebase SDK on the data side, and the separate "Secure Backend API" branch that reaches Gemini, OpenAI and the Agora token service.

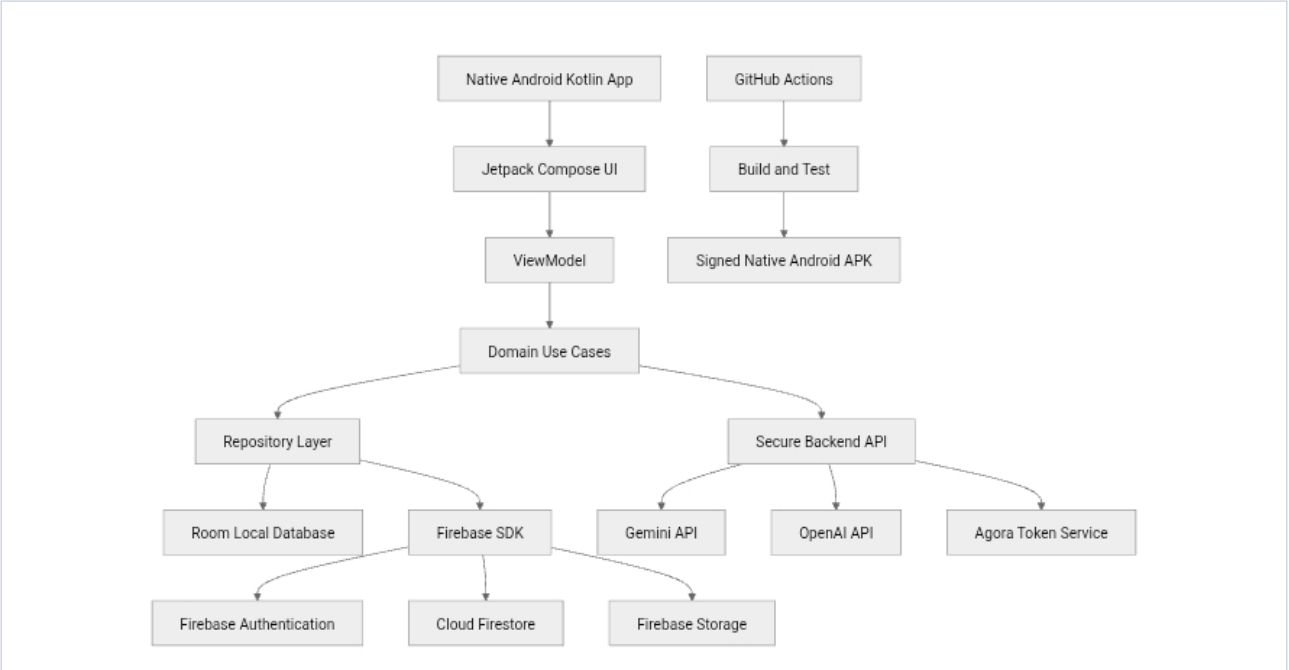


Figure 4.2 - Component view: Compose UI to ViewModel to use cases, with Firebase SDK and a secure backend for AI and live-class tokens; GitHub Actions produces the signed native APK.

4.1 Trust boundaries

Zone	Contains	Trust level	Crossing rule
Device	APK, Compose UI, Room cache, DataStore	Untrusted (user-controlled)	Everything is re-validated by rules or functions
Firebase client edge	Auth, Firestore, Storage, FCM endpoints	Semi-trusted	Rules + App Check + signed-in identity
Cloud Functions	TypeScript functions, scheduler jobs	Trusted	Verify auth, claims, quotas, then act with service account
Secret Manager	Gemini, OpenAI, Agora, payment secrets	Highly trusted	Only function service accounts may read
Third parties	Gemini, OpenAI, Agora, payment gateway	External	Called only from functions; responses sanitised

4.2 Google Cloud view

The next diagram, also supplied with the brief, shows how an actor reaches the application and how the application uses Authentication, Cloud Storage, Firestore and an AI-logic component inside a single Google Cloud project. The same grouping (Authentication, Application, Data Storage, AI Logic) is used in this blueprint.

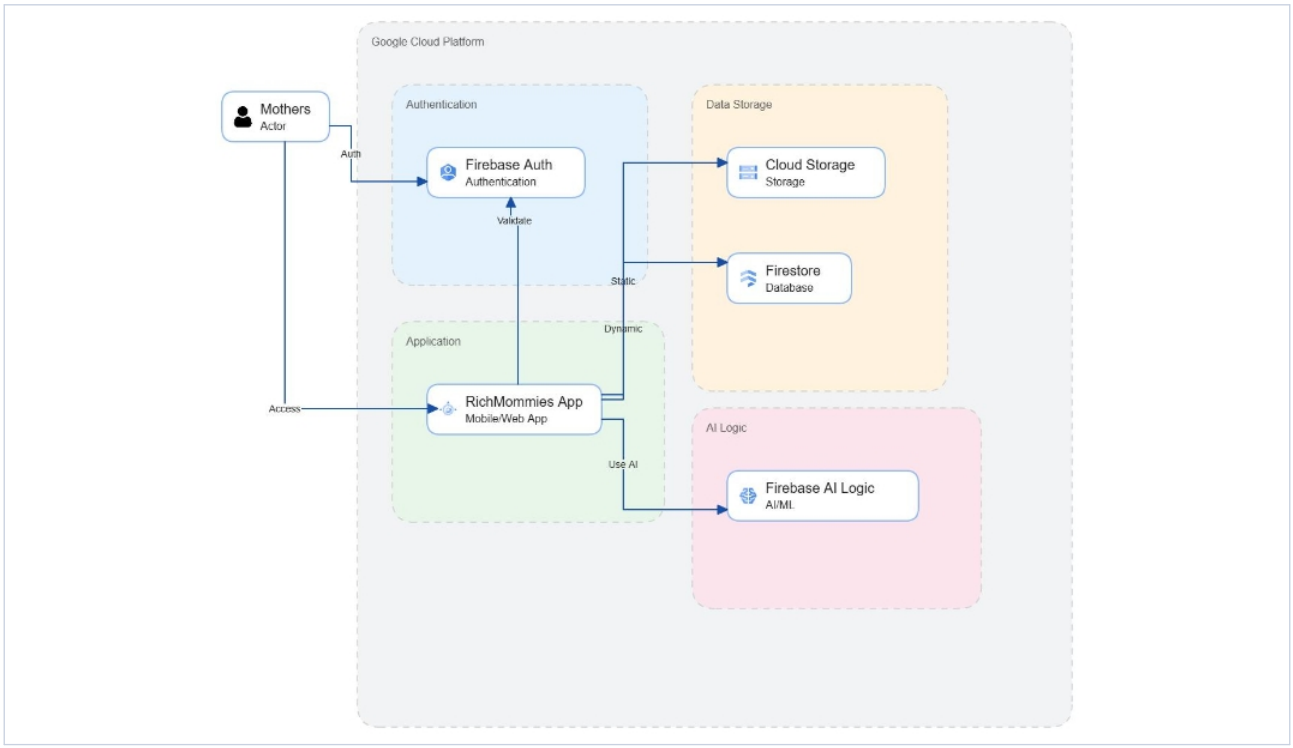


Figure 4.3 - Reference deployment grouping on Google Cloud: authentication, application, data storage and AI logic.

5. Android Application Architecture

The walkthrough video explains what happens inside the app for one user action: a UI screen reacts to a tap, a ViewModel validates state, a use case applies business rules, a repository coordinates sources, a data source talks to the cloud, and the result flows back as state.

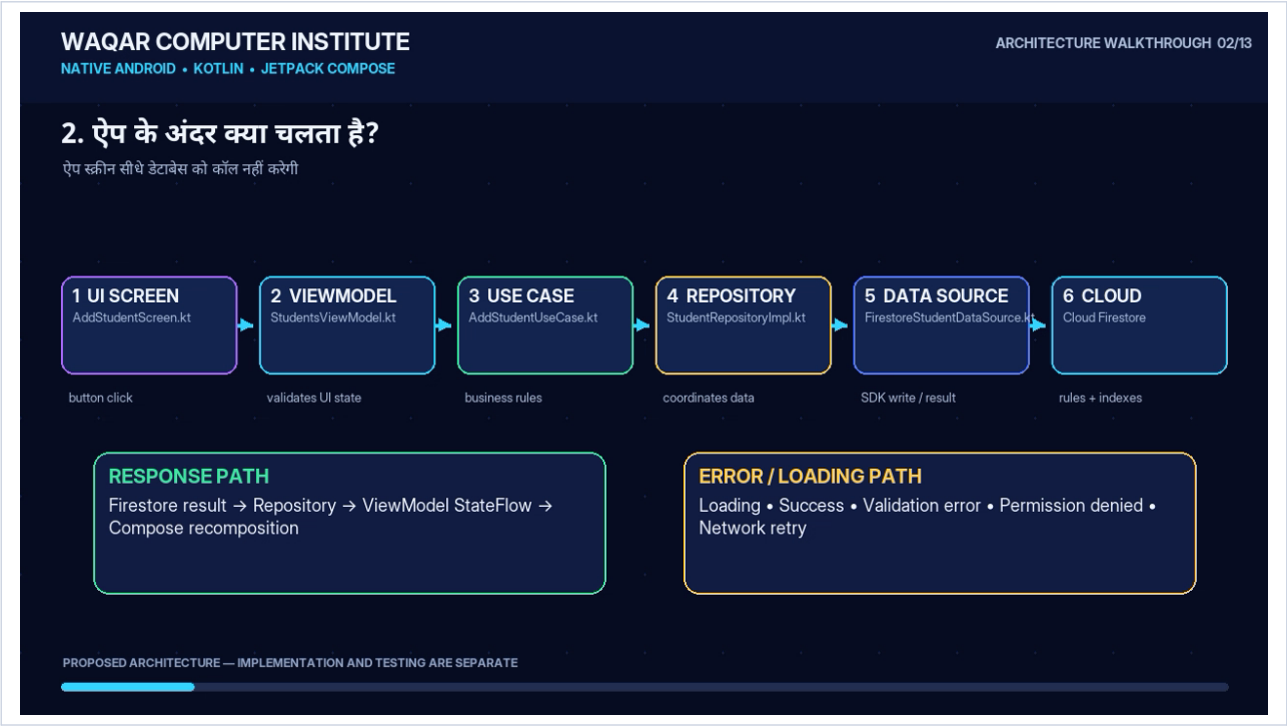


Figure 5.1 - Inside the app: UI screen, ViewModel, use case, repository, data source, cloud (video slide 2).

5.1 MVVM + Clean layering

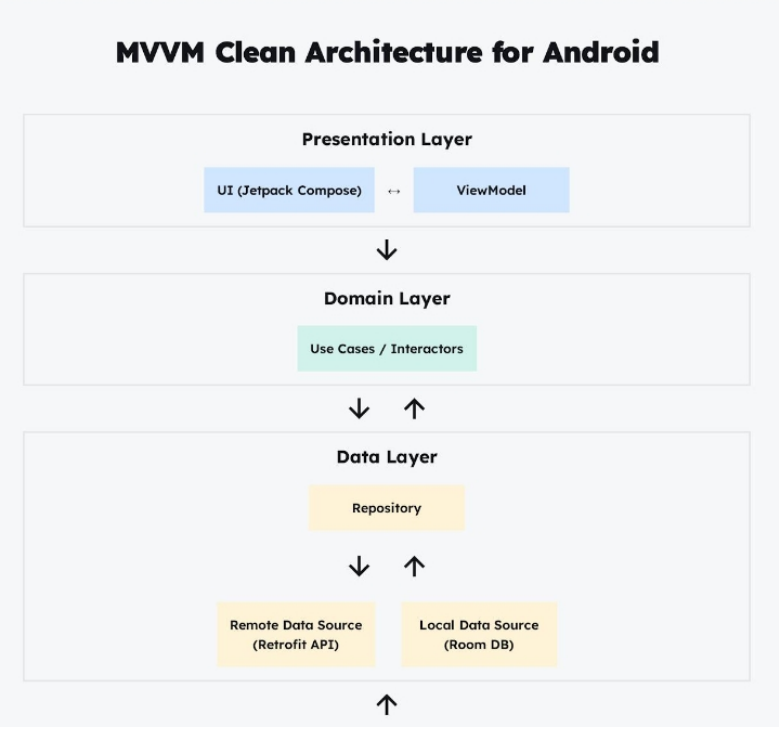


Figure 5.2 - MVVM Clean Architecture: presentation, domain and data layers (reference diagram). In this project the remote source is the Firebase SDK and callable functions, and the local source is Room.

The rule that holds the design together is the **dependency rule**: source-code dependencies point inward toward the domain layer. Control flows downward at run time, but the domain layer never imports Android, Compose or Firebase classes. The next diagram expresses this together with the SOLID principles.



Figure 5.3 - Clean Architecture and SOLID: calls flow down, dependencies point inward to the domain layer (reference diagram).

5.2 Dependency injection with Hilt

Hilt provides compile-time injection. An application component owns singletons such as the Firebase clients, the Room database and the dispatcher provider. Feature screens receive ViewModels through @HiltViewModel. The reference diagram below shows the same idea in classic Dagger form: an AppComponent with subcomponents for registration, user and login scopes. Hilt generates the equivalent wiring, so developers write modules, not components.

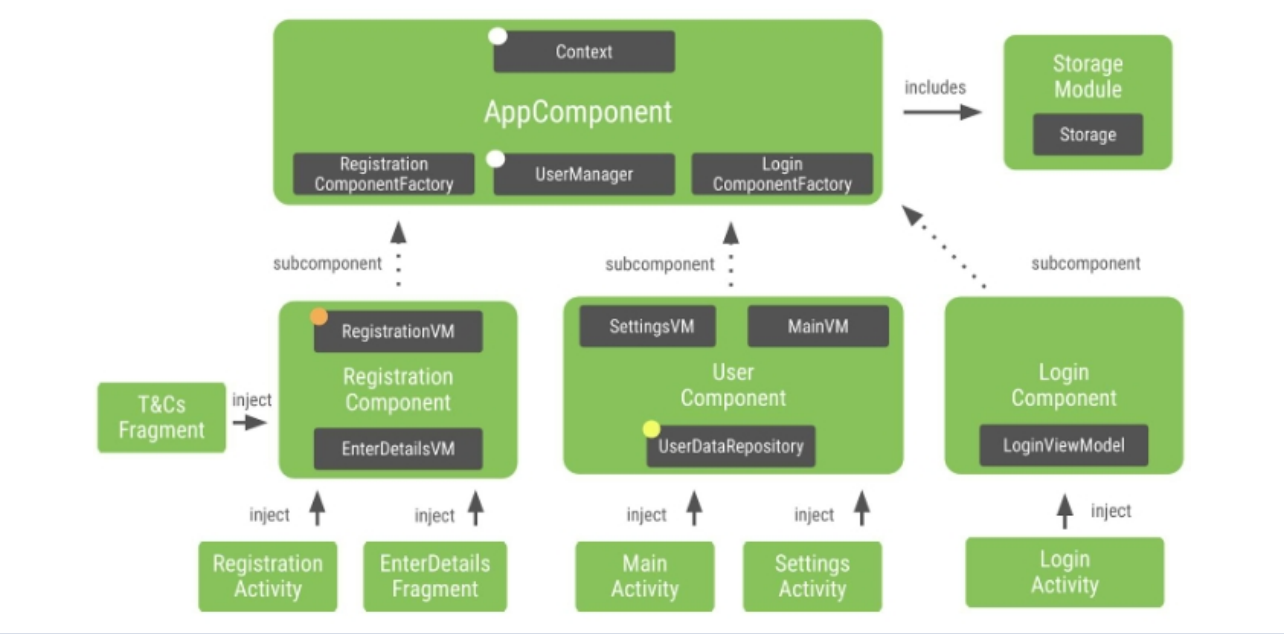


Figure 5.4 - Component and subcomponent scopes (Dagger reference diagram); Hilt generates the equivalent graph.

5.3 Gradle modules

Module	Contains	Depends on
:app	Application class, MainActivity, root NavHost, Hilt entry point	all feature modules, :core:*
:core:ui	Design system: glass, liquid, 3D components, theme, motion	Compose only
:core:domain	Entities, value objects, repository interfaces, base use cases	Kotlin only

Module	Contains	Depends on
:core:data	Firestore helpers, Room database, DataStore, mappers, paging sources	:core:domain
:core:network	Callable-function client, error mapping, optional pinned OkHttp	:core:domain
:core:security	Keystore/Tink wrapper, integrity, biometric, secure-screen helpers	Android only
:feature:auth	:feature:dashboard, :feature:students, :feature:attendance, :feature:fees, :feature:inbox	core modules
:feature:academics	:feature:admissions, :feature:exams, :feature:material, :feature:certificates, :feature:live, :feature:ai	core modules
:feature:operations	:feature:staff, :feature:payroll, :feature:accounts, :feature:inventory, :feature:support, :feature:reports	core modules
:feature:system	:feature:security, :feature:backup, :feature:settings	core modules

5.4 Package layout of one feature

```
feature/students/
  presentation/
    StudentsScreen.kt            // Compose list with paging
    StudentsViewModel.kt         // UiState + events (StateFlow / SharedFlow)
    AddStudentScreen.kt, StudentDetailScreen.kt
  domain/
    model/Student.kt
    usecase/AddStudentUseCase.kt, SearchStudentsUseCase.kt, ArchiveStudentUseCase.kt
    repository/StudentRepository.kt       // interface
  data/
    StudentRepositoryImpl.kt              // coordinates sources
    remote/FirestoreStudentDataSource.kt  // queries, writes, listeners
    local/StudentDao.kt, StudentEntity.kt // Room cache of recently viewed
    mapper/StudentMappers.kt
```

5.5 One vertical slice in Kotlin

The slice below shows how the layers connect for adding a student. It is an implementation pattern, not sample data: it contains no student values.

```
// domain
interface StudentRepository {
    suspend fun add(draft: StudentDraft): Result<StudentId>
    fun page(query: StudentQuery): Flow<PagingData<Student>>
}
class AddStudentUseCase @Inject constructor(private val repo: StudentRepository) {
    suspend operator fun invoke(d: StudentDraft): Result<StudentId> =
        StudentValidator.validate(d).fold(
            onFailure = { Result.failure(it) },
            onSuccess = { repo.add(d) })
}
// presentation
@HiltViewModel
class AddStudentViewModel @Inject constructor(private val addStudent: AddStudentUseCase) : ViewModel() {
    private val _state = MutableStateFlow(AddStudentState())
    val state: StateFlow<AddStudentState> = _state.asStateFlow()
    fun submit() = viewModelScope.launch {
        _state.update { it.copy(saving = true, error = null) }
        addStudent(_state.value.toDraft())
            .onSuccess { _state.update { s -> s.copy(saving = false, savedId = it) } }
            .onFailure { e -> _state.update { s -> s.copy(saving = false, error = e.toUiError()) } }
    }
}
```

5.6 State and error model

- **UiState** is an immutable data class per screen: loading, content, empty, error. Empty is a real state, never filled with sample rows.
- **One-shot events** (navigation, snackbars) use a SharedFlow so they are not replayed on rotation.
- **Errors** map to a sealed type: Network, PermissionDenied, Validation, NotFound, QuotaExceeded, Unknown. Each has a user-safe message and a diagnostics code for Crashlytics.
- **Dispatchers** are injected so tests can control time and threads.

6. Authentication, Roles and the Administrator Account

Login is username plus password. Behind the screen sit an authentication repository, the Firebase Authentication SDK and a server-controlled authorisation layer. The video slide below shows the four stages; the design choice it flags - username login needs a deliberate identity mapping - is resolved in section 6.2.

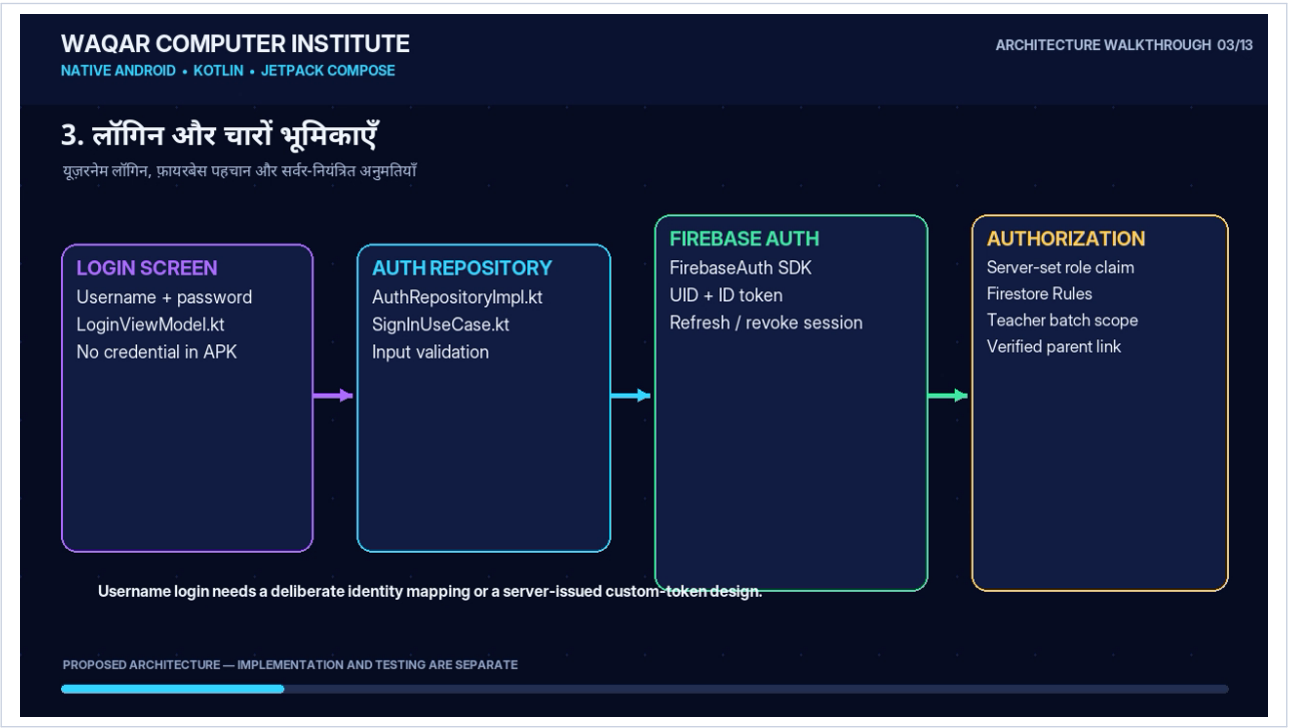


Figure 6.1 - Login screen, auth repository, Firebase Auth and authorisation (video slide 3).

6.1 Roles

Role	Created by	Capability summary	Claim
Admin (owner)	One-time bootstrap script	Everything, including security, backup, settings and role assignment; MFA mandatory	role=admin
Teacher	Admin invite	Own batches: attendance, marks, material, live classes, AI tutor	role=teacher, batches=[ids]
Student	Admin admission flow	Own timetable, attendance, fees, results, material, AI tutor, live classes	role=student, studentId
Parent	Admin guardian link + phone OTP	Read-only view of linked students: attendance, fees, results, notices	role=parent, children=[ids]

6.2 Username login design

Firebase Authentication signs users in with an email or phone identity. To keep a "username + password" screen, each username maps to a deterministic internal alias, for example `waqar@id.waqarinstitute.internal`. The alias is never shown to users and does not receive mail. The app lowercases the username, builds the alias, and calls the standard email-and-password sign-in. No lookup endpoint is needed, which avoids exposing a username directory.

- A public-safe document `usernames/{username}` reserves the name and stores the UID, so duplicates are impossible.
- Changing a username is a callable function (`changeUsername`): it checks the caller, updates the Auth alias through the Admin SDK, rewrites the reservation in a transaction, revokes other sessions and writes an audit entry.

- Forgot-password uses a real verified recovery email stored on the profile; the reset email goes to that address, not the alias.

6.3 Where the administrator credentials live

Item	Stored in	Why
Username (login name)	Firebase Auth alias + Firestore reservation	Allows unique login and instant in-app change
Password	Firebase Authentication only (salted hash, managed by Google)	Never readable by anyone, including the owner and developers
Profile (display name, phone, recovery email)	Firestore `staff/{uid}`	Editable in the app, synced instantly
Role and MFA state	Auth custom claims + Auth MFA enrolment	Server-controlled; cannot be edited by the client
Initial password	Entered once by the owner during bootstrap; never in the repo, chat or CI	Prevents a default credential from existing

Important. Do not write the real administrator password into this document, the repository, GitHub secrets or any chat. The bootstrap script (section 6.5) asks for it interactively on the owner's machine.

6.4 Changing credentials from inside the app

Step	Password change	Username change
1	Admin opens Profile > Security	Admin opens Profile > Security
2	App demands fresh re-authentication (password or biometric + password)	Same re-authentication
3	Client calls Firebase Auth updatePassword	Client calls changeUsername callable
4	Auth invalidates old refresh tokens; app signs in again	Function updates alias, reservation, claims; revokes tokens
5	Audit entry written by a function triggered on the event	Audit entry written in the same transaction
6	Push and email alert: "Password changed on this account"	Push and email alert: "Username changed"

6.5 One-time bootstrap script (runs on the owner's computer)

```
// scripts/bootstrap-admin.ts  (run once: npx ts-node scripts/bootstrap-admin.ts)
import { initializeApp, applicationDefault } from 'firebase-admin/app';
import { getAuth } from 'firebase-admin/auth';
import { getFirestore, FieldValue } from 'firebase-admin/firestore';
import * as readline from 'readline/promises';

const USERNAME = 'waqar';                       // login name chosen by the owner
const alias = `${USERNAME}@id.waqarinstitute.internal`;
async function main() {
  initializeApp({ credential: applicationDefault() });
  const rl = readline.createInterface({ input: process.stdin, output: process.stdout });
  const password = await rl.question('Choose admin password (min 14 chars): ');  // never stored on disk
  const recovery = await rl.question('Recovery email: ');
  rl.close();
  if (password.length < 14) throw new Error('Password too short');
  const user = await getAuth().createUser({ email: alias, password, displayName: 'Waqar' });
  await getAuth().setCustomUserClaims(user.uid, { role: 'admin' });
  const db = getFirestore();
  await db.runTransaction(async (t) => {
    t.create(db.doc(`usernames/${USERNAME}`), { uid: user.uid, createdAt: FieldValue.serverTimestamp() });
    t.create(db.doc(`staff/${user.uid}`), { role: 'admin', username: USERNAME, recoveryEmail: recovery,
      createdAt: FieldValue.serverTimestamp() });
  });
  console.log('Admin created. Enrol TOTP on first sign-in.');
}
main();
```

6.6 Sign-in sequence

#	Actor	Action
1	App	Validate input, lowercase username, build alias
2	App	signInWithEmailAndPassword(alias, password) through Firebase Auth
3	Auth	On success returns UID and ID token; if MFA is enrolled, returns a challenge
4	App	Collects TOTP code; completes sign-in
5	App	Reads token claims; routes to the portal that matches role

#	Actor	Action
6	App Check	Attaches a Play Integrity based token to every backend call
7	Function	Records login event; evaluates risk signals (new device, integrity verdict)
8	App	Starts Firestore listeners scoped to the role

7. Firebase and Google Cloud Backend

The next video slide answers the most important division-of-labour question: what does the Android SDK call directly, and what must a server do? Direct calls are acceptable only where Firestore and Storage rules can fully decide access. Everything involving a secret, money, bulk work or a third party is server-only.



Figure 7.1 - Android Firebase SDK versus server-only work, and the providers behind it (video slide 4).

7.1 Firebase product map



Figure 7.2 - Core Firebase components: build products (Firestore, Realtime Database, Authentication, Functions, Hosting-style storage) and run products (Crashlytics, Performance, Analytics, Remote Config, Messaging). Reference diagram.

Product	Used for	Used?
Authentication	Identity, MFA, custom claims	Yes
Cloud Firestore	Primary database (100k students and related records)	Yes

Product	Used for	Used?
Cloud Storage	Photos, ID proofs, materials, certificates, backups	Yes
Cloud Functions	Trusted logic, AI proxy, Agora tokens, payments, jobs	Yes
Cloud Messaging	Push notifications	Yes
App Check	Attest genuine app	Yes
Crashlytics / Performance / Analytics	Stability and speed monitoring	Yes
Remote Config	Feature flags and kill switches	Yes
Realtime Database	-	No (one primary database avoids split truth)
Hosting	-	No (no web app exists)

7.2 Client, backend and analytics paths

The SDK diagram below shows Android, iOS and web clients reaching Firebase, which fans out to Cloud Storage, Firestore and compute. This project uses only the Android column. The second diagram shows the mobile client with bi-directional sync to Firestore, Authentication, Messaging and Crashlytics, and an analytics export to BigQuery, which this blueprint keeps as an optional reporting toggle (function F374).

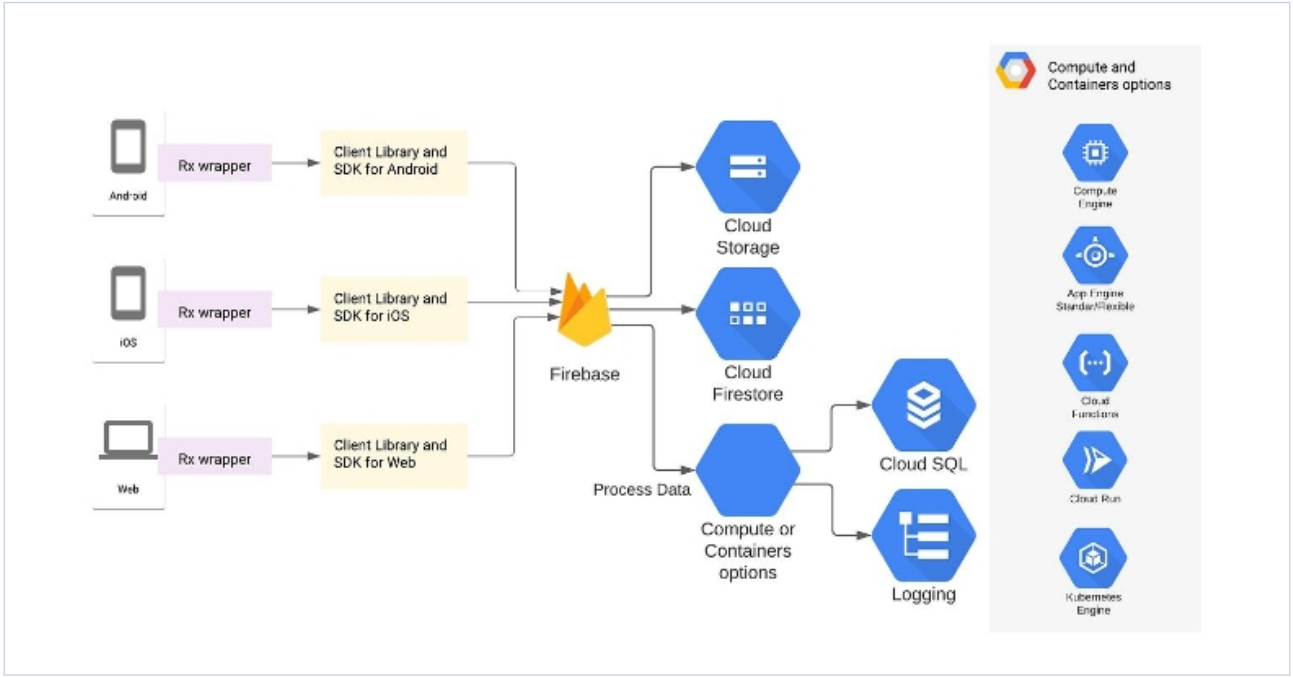


Figure 7.3 - Client SDKs to Firebase to storage, database and compute options (reference diagram).

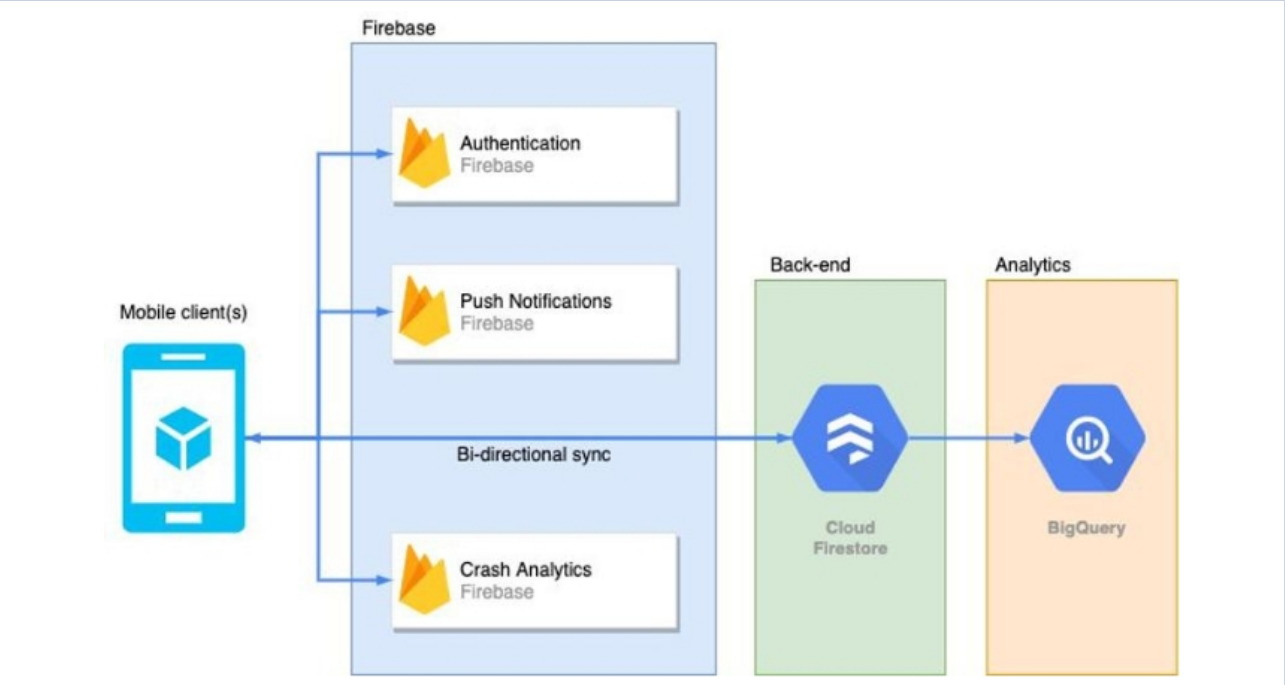


Figure 7.4 - Mobile client with Authentication, Push, Crash Analytics, bi-directional Firestore sync and BigQuery analytics (reference diagram).

7.3 Cloud Functions catalogue (server-only work)

Function	Trigger	Purpose
changeUsername	Callable	Atomic username change with alias update and token revocation
setUserRole	Callable (admin, four-eyes)	Set role claims and batch scopes
createStudent / importStudents	Callable / Storage-triggered	Enrolment-number transaction; CSV bulk import in 500-document batches
recordPayment / verifyPaymentWebhook	Callable / HTTPS	Server-side fee posting; signature-verified gateway webhook with idempotency
aiGemini / aiOpenAI	Callable	Authenticated, quota-checked proxy to the model providers
agoraToken	Callable	Mint short-lived Agora RTC token after role and batch check
sendNotification	Callable / Firestore trigger	Targeted FCM send, delivery log
onAuditWrite, onAttendanceWrite, onFeeWrite	Firestore triggers	Maintain aggregates, alerts and audit entries
nightlyExport, retentionSweep	Scheduler	Firestore export to locked bucket; apply retention rules
generateCertificate, verifyCertificate	Callable	Numbered PDF and public verification

Cloud Functions that call external providers need the Blaze (pay-as-you-go) plan. Set budget alerts on day one. Choose one Google Cloud region close to the institute and keep Firestore, Storage and Functions in the same region.

8. Firestore Data Model for 100,000 Students

Firestore scales horizontally by splitting data across servers and replicating each split across zones, as the reference diagram shows. The design consequence for this project: spread writes across many documents, avoid hot documents, and never read everything. The model below follows those rules.

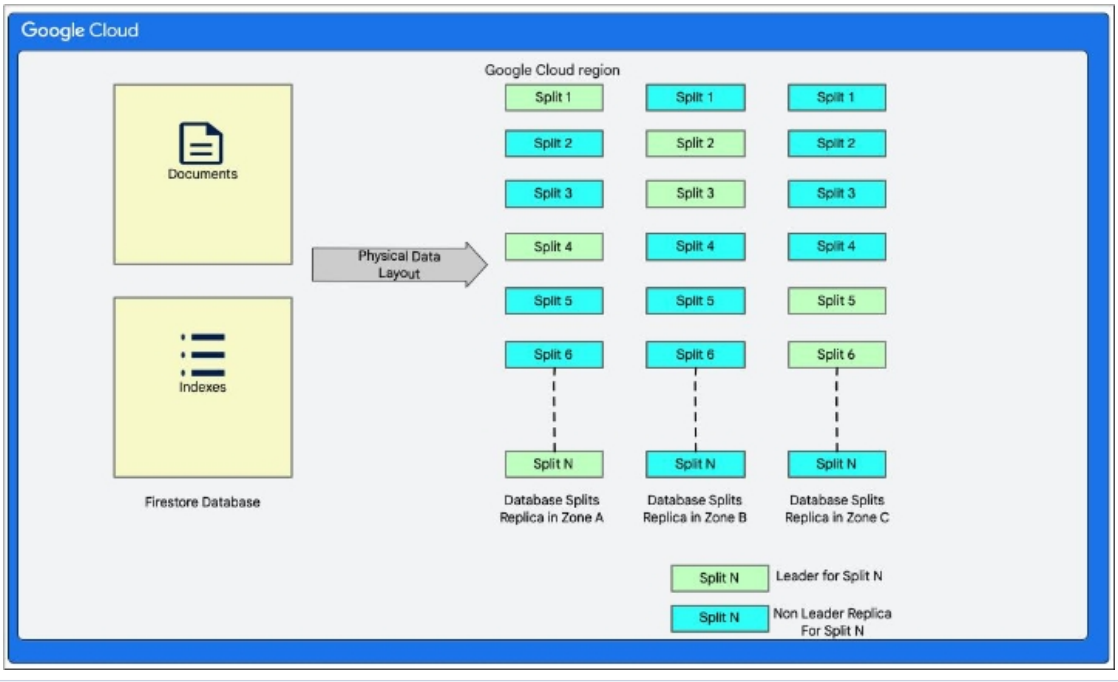


Figure 8.1 - Firestore physical layout: documents and indexes stored as splits, replicated across three zones with one leader per split (reference diagram).

8.1 Collections

Collection	Key	Main fields (types)	Notes
students	auto-ID	enrolmentNo (string), name, nameLower, phone, email, status, courseId, batchIds[], guardianIds[], feeState, photoPath, createdAt, updatedAt, createdBy	One document per student; about 100k+ documents. Sensitive documents live in a subcollection.
students/{id}/documents	auto-ID	kind, storagePath, uploadedBy, uploadedAt	ID proof and vault items
students/{id}/timeline	auto-ID	type, summary, actor, at	Capped by retention job
courses, batches	auto-ID	name, status, fee plan, capacity, schedule, trainerIds[]	Hundreds, not thousands
attendance	{batchId}_{yyyyMMdd}	batchId, date, marks map(studentId to code), lockedAt	One document per batch-day avoids per-student write storms
fees	auto-ID	studentId, planId, amount, due, paid, status, receiptNo, mode, at	Posted by server only
exams, results	auto-ID	examId, studentId, marks map, grade, published	Results published by function
staff	UID	role, username, profile, batchIds[]	Admin and teacher profiles
usernames	username	uid, createdAt	Reservation for unique login names
announcements, inbox/{uid}/items	auto-ID	title, body, target, scheduledAt; per-user read state	Fan-out by function
counters	name	value	Sharded when a counter needs more than about one write per second
auditLog	auto-ID	actor, action, targetPath, before, after, at, ip/device hash	Append-only; admin read only
aiUsage	{uid}_{yyyyMM}	requests, tokens, provider, lastAt	Quota enforcement and cost view

8.2 Search and list performance

- **Never load all students.** Lists use cursor paging of 25-50 documents with Paging 3 and Firestore query cursors.
- **Prefix search** uses an indexed `nameLower` range query (start-at / end-at with a high Unicode sentinel). Phone and enrolment number are exact-match queries.
- **Filters** (course, batch, status, fee state) are equality filters with composite indexes (8.3). Sort is by `createdAt` or `nameLower`.
- **Counts** on the dashboard use Firestore aggregation queries (count) and function-maintained counter documents, not document scans.
- **Full-text search** is out of scope for v1. If the owner later needs fuzzy search, add a dedicated search service behind a Cloud Function.

8.3 Composite indexes (firestore.indexes.json excerpt)

```
{ "indexes": [
  { "collectionGroup": "students", "queryScope": "COLLECTION",
    "fields": [ {"fieldPath":"courseId","order":"ASCENDING"}, {"fieldPath":"nameLower","order":"ASCENDING"} ] },
  { "collectionGroup": "students", "queryScope": "COLLECTION",
    "fields": [ {"fieldPath":"status","order":"ASCENDING"}, {"fieldPath":"createdAt","order":"DESCENDING"} ] },
  { "collectionGroup": "students", "queryScope": "COLLECTION",
    "fields": [ {"fieldPath":"batchIds","arrayConfig":"CONTAINS"}, {"fieldPath":"nameLower","order":"ASCENDING"} ] },
  { "collectionGroup": "students", "queryScope": "COLLECTION",
    "fields": [ {"fieldPath":"feeState","order":"ASCENDING"}, {"fieldPath":"nameLower","order":"ASCENDING"} ] },
  { "collectionGroup": "fees", "queryScope": "COLLECTION",
    "fields": [ {"fieldPath":"studentId","order":"ASCENDING"}, {"fieldPath":"due","order":"DESCENDING"} ] }
] }
```

8.4 Security rules (excerpt)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{db}/documents {
    function signedIn()  { return request.auth != null; }
    function role()      { return request.auth.token.role; }
    function isAdmin()   { return signedIn() && role() == 'admin'; }
    function isTeacher() { return signedIn() && role() == 'teacher'; }
    function validStudent(d) {
      return d.name is string && d.name.size() > 1 && d.name.size() <= 120
          && d.nameLower == d.name.lower()
          && d.phone is string && d.phone.size() >= 8 && d.phone.size() <= 15
          && d.status in ['active','archived','alumni']
          && d.keys().hasOnly(['enrolmentNo','name','nameLower','phone','email','status','courseId',
             'batchIds','guardianIds','feeState','photoPath','createdAt','updatedAt','createdBy']);
    }
    match /students/{id} {
      allow read:   if isAdmin()
                    || (isTeacher() && resource.data.batchIds.hasAny(request.auth.token.batches))
                    || (role() == 'student' && request.auth.token.studentId == id)
                    || (role() == 'parent'  && id in request.auth.token.children);
      allow create: if isAdmin() && validStudent(request.resource.data);
      allow update: if isAdmin() && validStudent(request.resource.data);
      allow delete: if false;                     // soft delete via status only
    }
    match /fees/{id}      { allow read: if isAdmin(); allow write: if false; }   // server only
    match /auditLog/{id}  { allow read: if isAdmin(); allow write: if false; }   // append by function
    match /{document=**}  { allow read, write: if false; }                        // deny by default
  }
}
```

8.5 Bulk import of existing records

If the institute already has records, import them by uploading a CSV to a restricted Storage path. A Storage-triggered function streams the file, validates each row with the same schema as the rules, writes through a BulkWriter in controlled batches, and produces a result file listing accepted and rejected rows. Nothing is invented: rows that fail validation are reported, never auto-corrected.

8.6 Capacity and limits to respect

Concern	Design response
Sustained writes to a single document	Keep below roughly one write per second per document; shard counters; one attendance document per batch-day
Sequential IDs and hot ranges	Use auto-IDs for documents; enrolment numbers are a field produced by a counter transaction
Document size	Keep documents small; move media to Storage and large lists to subcollections
Listener cost	Attach listeners only on visible screens and small result sets; paging uses one-time queries
Cost control	Aggregation queries, counters and report snapshots; budget alerts; measure reads per screen in staging

9. Student Save Flow, Realtime Sync and Offline Behaviour

The brief requires that anything the administrator saves is stored in the cloud immediately. The video traces one save end to end: form state, use case, repository, Firestore write, security-rule validation, listener update on other devices and an audit event for sensitive changes.

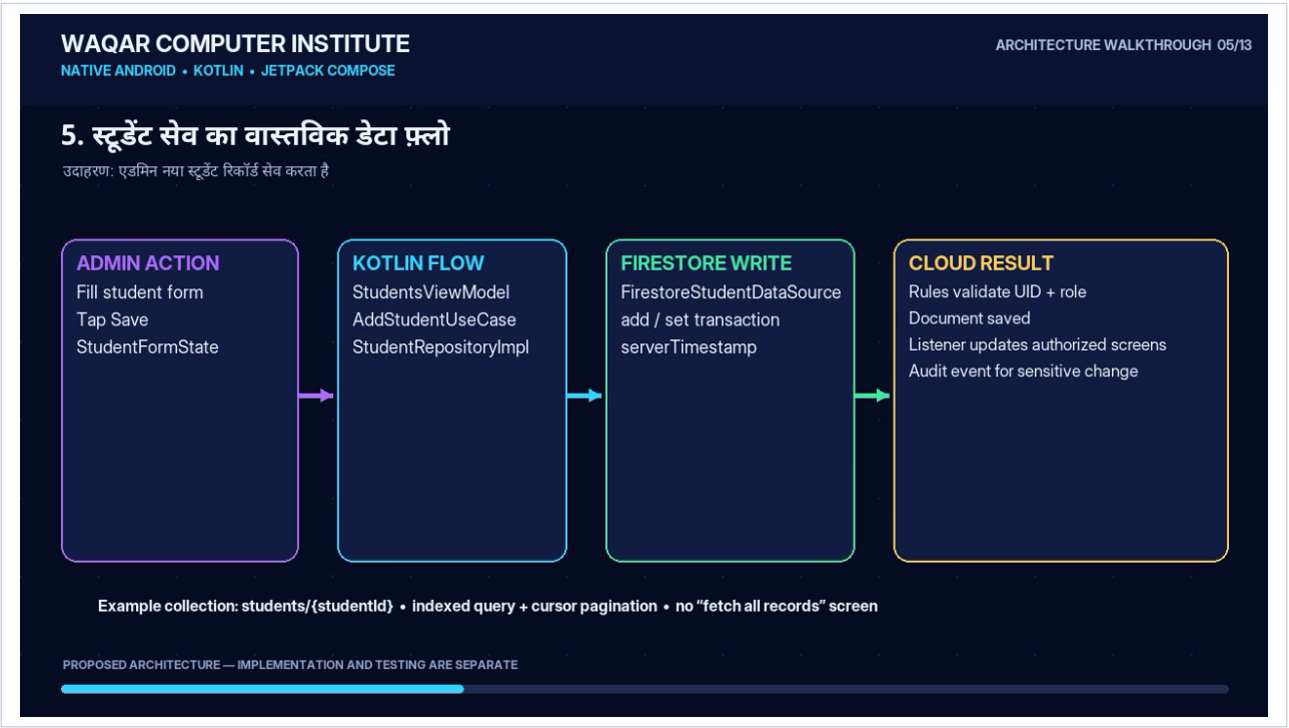


Figure 9.1 - Real data flow of a student save (video slide 5).

9.1 Write path

Step	Component	Detail
1	StudentFormState	Field-level validation (length, phone format, required course)
2	AddStudentUseCase	Business rules (duplicate phone check query, course capacity)
3	StudentRepositoryImpl	Requests an enrolment number via callable, then writes the student
4	Firestore SDK	Write is sent to the cloud with a server timestamp; local cache updates instantly
5	Security Rules	Validate role, schema and field set; reject anything else
6	Firestore trigger	Updates counters, writes audit entry for sensitive changes
7	Listeners on other screens/devices	Receive the new document and update the UI without refresh

9.2 Offline and sync

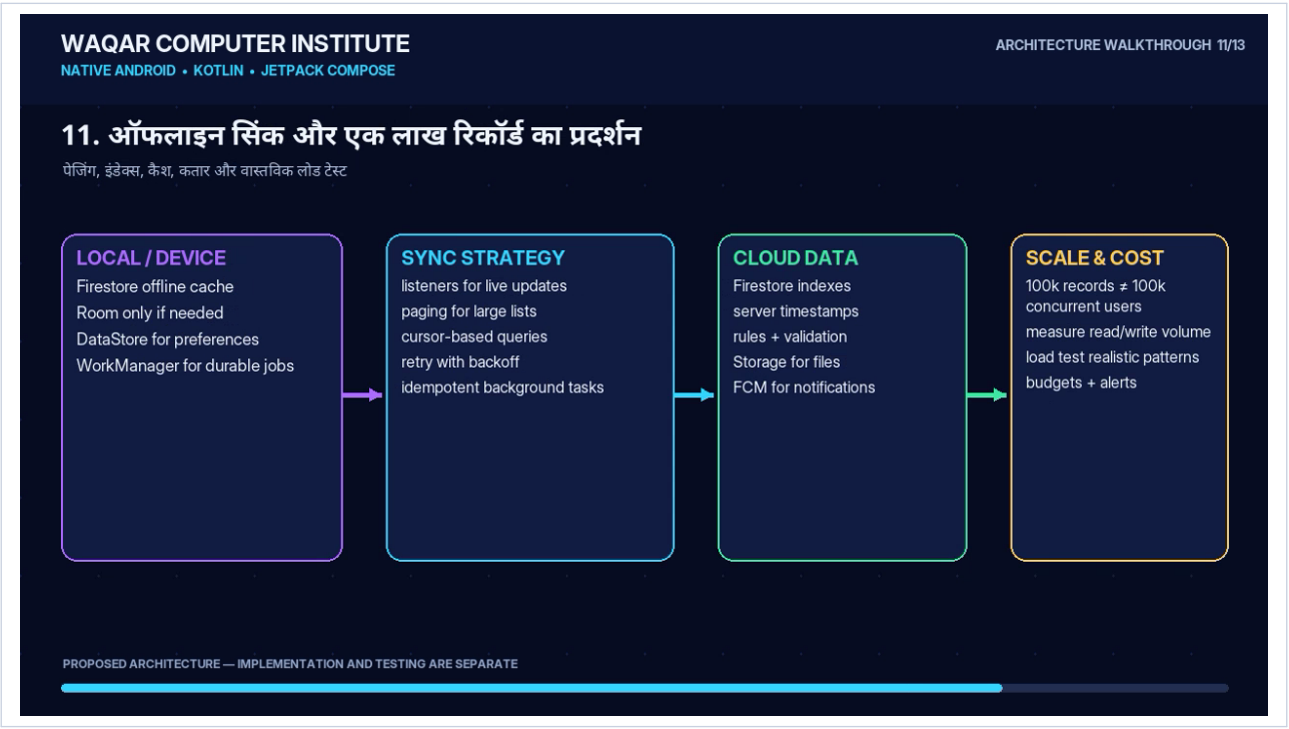


Figure 9.2 - Offline sync and performance for 100,000 records (video slide 11).

- **Firestore offline persistence** keeps recently used documents and queues writes while offline; queued writes are sent in order on reconnection.
- **Room** caches only what a screen needs often (recent students, today's timetable, unsent attendance). It does not mirror 100k records.
- **WorkManager** retries durable jobs (attendance queue, photo uploads) with exponential backoff and idempotent request IDs.
- **Conflict policy**: last write wins for profile fields; attendance and fee postings are server-arbitrated, so a stale device cannot overwrite them silently.
- **Staleness**: the UI shows a "data from cache" chip when Firestore reports metadata.fromCache.

9.3 Performance budgets

Metric	Budget	How verified
Cold start to first interactive frame	Under 2 s on a mid-range device	Macrobenchmark + Baseline Profile
Students list first page	Under 1 s on normal mobile data	Performance traces, staging project
Scrolling	No dropped frames at 60 Hz on list screens	Macrobenchmark frame timing
Reads per screen open	Documented and capped per screen	Firestore usage dashboard
APK/AAB size	Tracked per release; regressions reviewed	CI size report

10. Admin Portal Information Architecture

The administrator portal has two navigation systems that never duplicate each other. The **bottom tab bar** holds the five screens the owner uses many times a day. The **side menu** (navigation drawer on phones, navigation rail on tablets) holds everything else, grouped by business area. Every one of the 450 functions in Appendix A is either a screen reachable from one of these two places or an action inside such a screen.

10.1 Bottom tab bar (5 tabs)

Tab	Icon concept	Module	Why it is a tab
1 Home	Dashboard glass card	M02 Admin Dashboard	Daily health of the institute
2 Students	Group	M03 Student Management	Most frequent lookup and edit
3 Attendance	Check-calendar	M07 Attendance	Done every class, must be fast
4 Fees	Rupee wallet	M08 Fees & Payments	Daily cash handling
5 Inbox	Bell	M16 Notifications & Announcements	Approvals, alerts, messages

10.2 Side menu groups

Group	Destination	Module	Functions
People	Admissions	M04 Admissions & Enquiries	20
People	Staff	M13 Teacher & Staff Management	22
People	Guardians	M15 Guardian (Parent) Linking	12
Academics	Courses	M05 Course Catalog	18
Academics	Batches	M06 Batch & Timetable	22
Academics	Exams	M10 Examinations & Results	24
Academics	Material	M11 Assignments & Study Material	18
Academics	Certificates	M12 Certificates	14
Academics	Live Classes	M17 Live Classes (Agora)	20
Academics	AI Hub	M18 AI Tutor & Assistant	18
Finance	Accounts	M09 Accounts & Expenses	18
Finance	Payroll	M14 Payroll & Leave	16
Insights	Reports	M19 Reports & Analytics	22
Operations	Inventory	M20 Inventory, Lab & Library	14
Operations	Support	M21 Support & Feedback	12
System	Security	M22 Security & Audit	20
System	Backup	M23 Backup & Data Tools	14
System	Settings	M24 Settings & Institute Profile	16
Account	Profile menu (avatar): password, username, sessions, sign out	M01 Authentication & Session	18

10.3 Function types and where they appear

Type	Meaning	Placement rule
Screen (S)	A full destination	Appears in tab bar or side menu exactly as listed in Appendix A
Action (A)	A button, sheet or dialog	Inside its parent screen; never a separate menu item
List / filter (L)	Query, filter chip or search field	Inside list screens; filters collapse into a glass bottom sheet
Report / export (R)	Chart, table or file output	Reports module plus a "share/export" button on the source screen
Background job (B)	Automatic work	No UI except status chips and notifications
Configuration (C)	Setting or rule	Settings group, with deep links from the screen it affects

10.4 Module summary

Module	Navigation placement	Roles	Functions
M01 Authentication & Session	Pre-login flow + Profile menu (top-right avatar)	All roles	18
M02 Admin Dashboard	TAB 1: Home	Admin	16
M03 Student Management	TAB 2: Students	Admin (write); Teacher (read, own batches)	30
M04 Admissions & Enquiries	Drawer > People > Admissions	Admin	20
M05 Course Catalog	Drawer > Academics > Courses	Admin (write); Teacher (read)	18
M06 Batch & Timetable	Drawer > Academics > Batches	Admin (write); Teacher (own batches); Student/Parent (read own)	22
M07 Attendance	TAB 3: Attendance	Admin; Teacher (own batches); Student/Parent (read own)	20
M08 Fees & Payments	TAB 4: Fees	Admin (write); Parent/Student (read own)	28
M09 Accounts & Expenses	Drawer > Finance > Accounts	Admin	18
M10 Examinations & Results	Drawer > Academics > Exams	Admin; Teacher (own batches); Student/Parent (read own)	24
M11 Assignments & Study Material	Drawer > Academics > Material	Admin; Teacher (write); Student (read assigned)	18
M12 Certificates	Drawer > Academics > Certificates	Admin; Student (read own)	14
M13 Teacher & Staff Management	Drawer > People > Staff	Admin	22
M14 Payroll & Leave	Drawer > Finance > Payroll	Admin; Staff (own payslip, leave)	16
M15 Guardian (Parent) Linking	Drawer > People > Guardians	Admin; Parent (own profile)	12
M16 Notifications & Announcements	TAB 5: Inbox	Admin (send); Teacher (send to own batches); all (receive)	18
M17 Live Classes (Agora)	Drawer > Academics > Live Classes	Admin; Teacher (host); Student (join assigned)	20
M18 AI Tutor & Assistant	Drawer > Academics > AI Hub	Student/Teacher (tutor); Admin (assistant, config)	18
M19 Reports & Analytics	Drawer > Insights > Reports	Admin; Teacher (own batches, limited)	22
M20 Inventory, Lab & Library	Drawer > Operations > Inventory	Admin	14
M21 Support & Feedback	Drawer > Operations > Support	Admin; Teacher; Student/Parent (create, read own)	12
M22 Security & Audit	Drawer > System > Security	Admin (owner) only	20
M23 Backup & Data Tools	Drawer > System > Backup	Admin (owner) only	14
M24 Settings & Institute Profile	Drawer > System > Settings	Admin (owner)	16
Total			450

11. Teacher, Student and Parent Portals

The app resolves the role claim after sign-in and loads the matching portal. The same functions from Appendix A are reused with narrower scopes; no function is duplicated.

Portal	Bottom tabs	Side menu
Teacher	Today (timetable), Attendance, Batches, Material, Inbox	Exams and marks, Live classes, AI tutor, Leave and payslip, Support, Profile
Student	Home, Timetable, Fees, Results, Inbox	Attendance, Material and assignments, Live classes, AI tutor, Certificates, Support, Profile
Parent	Children, Attendance, Fees, Results, Inbox	Timetable, Notices, Message teacher, Consent and preferences, Support, Profile

11.1 Capability matrix (summary)

Capability	Admin	Teacher	Student	Parent
Create / edit students	Yes	No	No	No
View student record	All	Own batches	Self	Linked children
Mark attendance	Yes	Own batches	No	No
View attendance	All	Own batches	Self	Linked children
Record payments	Yes	No	No	No
View fees	All	No	Self	Linked children
Enter marks	Yes	Own batches	No	No
View results	All	Own batches	Self	Linked children
Send announcements	All	Own batches	No	No
Host live class	Yes	Yes	No	No
Join live class	Yes	Yes	Assigned	No
Use AI tutor	Admin assistant	Yes	Yes (quota)	No
Security, backup, settings	Yes	No	No	No

The matrix is enforced twice: the UI hides what a role cannot use, and Firestore Rules or Cloud Functions refuse it regardless of what the UI shows.

12. UI / UX Design System: Glass, Liquid and 3D

The visual target is a premium, bank-grade product: depth, translucency, fluid motion and clear hierarchy, without sacrificing speed. The design system lives in `:core:ui` so every feature uses identical components. Effects are tiered by device capability so the app is beautiful on flagship phones and still smooth on budget phones.

12.1 Visual language

Principle	Implementation
Glass surfaces	Translucent panels with backdrop blur, 1 dp luminous border and soft inner highlight; used for cards, bottom bar, drawer and sheets
Liquid motion	Spring-based shared-element transitions, metaball-style selection indicator in the tab bar, ripple shaders on press
3D depth	Layered parallax on dashboard cards, tilt response from the gyroscope, real-time 3D hero models on login and empty states
Typography	One variable sans family; strict scale (display, title, body, label); tabular numerals for money
Colour	Dynamic dark and light themes from a brand palette (navy, cyan, violet); semantic colours for fee states and attendance
Haptics and sound	Subtle haptic confirmation on save, error and swipe; no sound by default
Accessibility	Contrast checks on glass, font scaling to 200 percent, screen-reader labels, reduce-motion switch (F445)

12.2 Capability tiers

Tier	Device condition	Behaviour
Ultra	Android 13+ (AGSL), strong GPU, Android performance class	Full blur, shader ripples, 3D hero scenes, parallax
High	Android 12+ (RenderEffect blur)	Blur and spring motion, simplified 3D (static render)
Standard	Older or low-RAM devices	Translucent gradient surfaces with no blur, light animation
Reduced	User enables reduce motion or battery saver	Crossfades only; no parallax, no continuous shaders

The tier is chosen at start-up from API level, memory class and a short frame-time probe, and can be overridden in Settings (F444). Effects are never required to understand or operate a screen.

12.3 Glass panel pattern

```
@Composable
fun GlassCard(modifier: Modifier = Modifier, tier: UiTier = LocalUiTier.current,
              content: @Composable ColumnScope.() -> Unit) {
    val shape = RoundedCornerShape(24.dp)
    val surface = when (tier) {
        UiTier.Ultra, UiTier.High -> Modifier.hazeChild(state = LocalHazeState.current,
                                     style = HazeMaterials.thin())          // backdrop blur
        else -> Modifier.background(Brush.verticalGradient(                 // no-blur fallback
                    listOf(Color.White.copy(alpha = .14f), Color.White.copy(alpha = .06f))))
    }
    Column(modifier.clip(shape).then(surface)
        .border(1.dp, Color.White.copy(alpha = .22f), shape).padding(16.dp), content = content)
}
```

12.4 Library roles in the UI layer

Need	Library	Notes
Backdrop blur	Haze	Provides blur with a software fallback
Liquid / lens effect	Backdrop-style liquid glass library + AGSL shaders	Verify maintenance status before adoption; keep behind an interface so it can be replaced
3D	Filament / SceneView	Load compressed glTF models from assets; pause rendering off-screen
Motion	Compose animation, Lottie	Spring specs centralised in the theme
Charts	Vico	Dashboard and report charts
Images	Coil	Memory and disk cache with size caps

12.5 UI performance rules

- No more than two live blur layers on screen; blur regions are sized to the content, not the screen.
- 3D scenes render only while visible and drop to a static image on thermal throttling.
- Lists use stable keys, immutable state and lazy layouts; glass is applied to rows only when the row count is small.
- Baseline Profiles ship with every release; Macrobenchmark guards startup and scrolling in CI.

13. AI, Live Classes and Secret Management

Gemini, OpenAI and Agora are the three external providers. The rule is absolute: **no provider secret is ever placed in BuildConfig, strings.xml, resources, assets, Git history, GitHub variables or the APK**. The app authenticates the user to Firebase; Cloud Functions authenticate to the providers.

13.1 Where each secret lives

Secret	Stored in	Read by	Never in
Gemini API key	Google Secret Manager (GEMINI_API_KEY)	aiGemini function	APK, repo, GitHub, logs
OpenAI API key	Google Secret Manager (OPENAI_API_KEY)	aiOpenAI function	APK, repo, GitHub, logs
Agora App Certificate	Google Secret Manager (AGORA_APP_CERTIFICATE)	agoraToken function	APK, repo, GitHub, logs
Agora App ID	Remote Config or function response (identifier, not secret)	App at join time	-
Payment gateway secret and webhook secret	Google Secret Manager	Payment functions	APK, repo, GitHub
Android signing keystore	GitHub encrypted secret (Base64) and an offline backup	Release workflow only	Repo files

13.2 Secure AI connection



Figure 13.1 - AI tutor screen to backend proxy to Cloud Function to Gemini / OpenAI (video slide 6).

Request path: the AI tutor screen sends the message to its repository; the repository calls a callable function with the user's Firebase identity and App Check token; the function verifies role, quota and safety rules, retrieves the secret from Secret Manager, calls the provider, filters the answer, records usage and returns text (or a stream).

```
// functions/src/ai/aiProxy.ts
import { onCall, HttpsError } from 'firebase-functions/v2/https';
import { defineSecret, defineString } from 'firebase-functions/params';
import { getFirestore, FieldValue } from 'firebase-admin/firestore';

const GEMINI_KEY = defineSecret('GEMINI_API_KEY');
const OPENAI_KEY = defineSecret('OPENAI_API_KEY');
const GEMINI_MODEL = defineString('GEMINI_MODEL');     // set per environment, not hard-coded
const OPENAI_MODEL = defineString('OPENAI_MODEL');
```

```
export const aiChat = onCall(
  { secrets: [GEMINI_KEY, OPENAI_KEY], enforceAppCheck: true, region: 'asia-south1', maxInstances: 20 },
  async (req) => {
    if (!req.auth) throw new HttpsError('unauthenticated', 'Sign in required');
    const role = req.auth.token.role as string;
    if (!['student', 'teacher', 'admin'].includes(role)) throw new HttpsError('permission-denied', 'Not allowed');
    const { prompt, provider } = req.data as { prompt: string; provider?: 'gemini' | 'openai' };
    if (typeof prompt !== 'string' || prompt.length === 0 || prompt.length > 4000)
      throw new HttpsError('invalid-argument', 'Prompt length invalid');
    await enforceQuota(req.auth.uid, role);                        // per-user monthly + per-minute limits
    const text = provider === 'openai' ? await callOpenAI(prompt) : await callGemini(prompt);
    await getFirestore().doc(`aiUsage/${req.auth.uid}_${yyyymm()}`)
      .set({ requests: FieldValue.increment(1), lastAt: FieldValue.serverTimestamp() }, { merge: true });
    return { text: sanitize(text) };
  });

async function callGemini(prompt: string): Promise<string> {
  const r = await fetch(`https://generativelanguage.googleapis.com/v1beta/models/${GEMINI_MODEL.value()}:generateContent`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json', 'x-goog-api-key': GEMINI_KEY.value() },
    body: JSON.stringify({ contents: [{ parts: [{ text: prompt }] }] }) });
  if (!r.ok) throw new HttpsError('unavailable', 'AI provider error');
  const j: any = await r.json();
  return j.candidates?.[0]?.content?.parts?.map((p: any) => p.text).join('') ?? '';
}
async function callOpenAI(prompt: string): Promise<string> {
  const r = await fetch('https://api.openai.com/v1/responses', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json', Authorization: `Bearer ${OPENAI_KEY.value()}` },
    body: JSON.stringify({ model: OPENAI_MODEL.value(), input: prompt }) });
  if (!r.ok) throw new HttpsError('unavailable', 'AI provider error');
  const j: any = await r.json();
  return j.output_text ?? '';
}
```

The model names come from environment parameters so they can be changed without a new APK and are checked against the provider documentation at implementation time. Provider fallback (F340) retries the other provider on a 5xx or timeout.

13.3 Agora voice and video

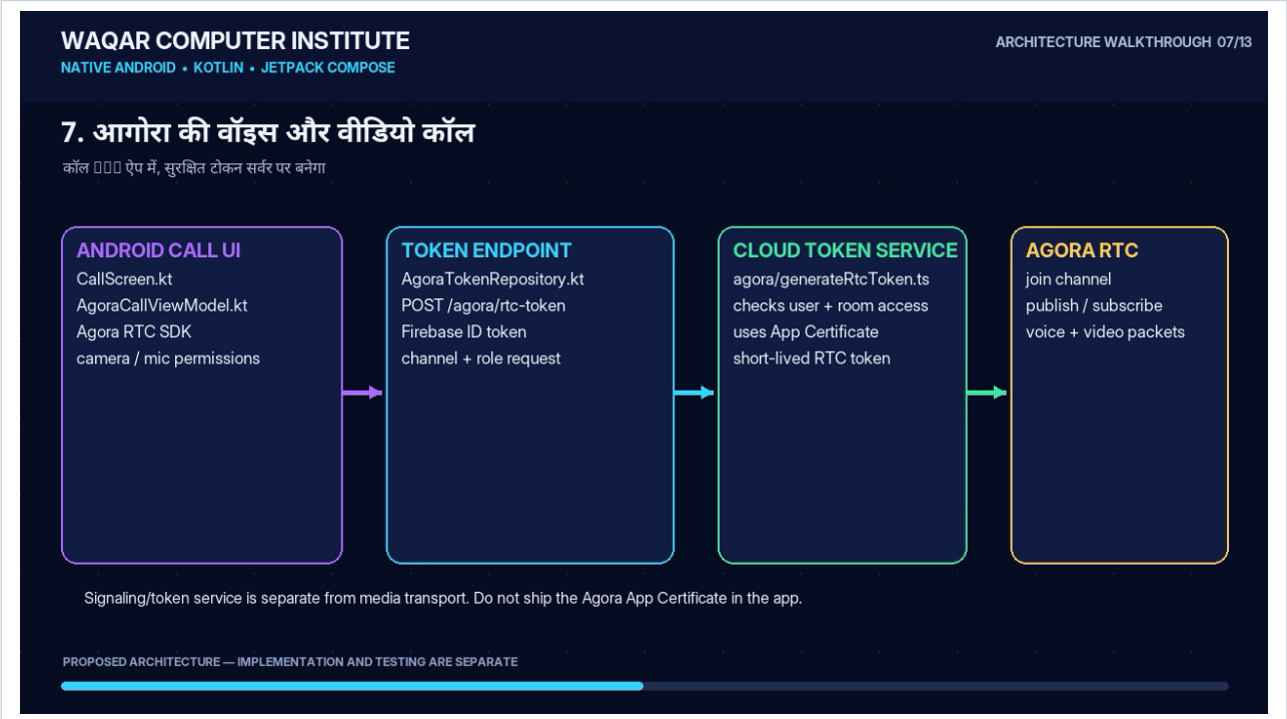


Figure 13.2 - Call UI, token endpoint, cloud token service and Agora RTC (video slide 7).

The app never holds the Agora App Certificate. Before joining, it asks the `agoraToken` function for a token. The function checks that the caller is signed in, that the class exists, and that the caller is the host or an assigned student; it then mints a token valid for a short time for that user and that channel. Media flows directly between the

device and Agora; only signalling and token requests touch our backend.

```
// functions/src/live/agoraToken.ts
import { onCall, HttpsError } from 'firebase-functions/v2/https';
import { defineSecret, defineString } from 'firebase-functions/params';
import { RtcTokenBuilder, RtcRole } from 'agora-token';
import { getFirestore } from 'firebase-admin/firestore';

const CERT = defineSecret('AGORA_APP_CERTIFICATE');
const APP_ID = defineString('AGORA_APP_ID');

export const agoraToken = onCall({ secrets: [CERT], enforceAppCheck: true, region: 'asia-south1' }, async (req) => {
  if (!req.auth) throw new HttpsError('unauthenticated', 'Sign in required');
  const { classId } = req.data as { classId: string };
  const snap = await getFirestore().doc(`liveClasses/${classId}`).get();
  if (!snap.exists) throw new HttpsError('not-found', 'Class not found');
  const c = snap.data()!; const t = req.auth.token;
  const isHost = c.hostUid === req.auth.uid;
  const allowed = isHost || t.role === 'admin' || (t.role === 'student' && c.batchId && t.batches?.includes?.(c.batchId));
  if (!allowed) throw new HttpsError('permission-denied', 'Not assigned to this class');
  const ttl = 60 * 30;                                    // 30 minutes
  const uid = hashToUint32(req.auth.uid);
  const token = RtcTokenBuilder.buildTokenWithUid(APP_ID.value(), CERT.value(), classId, uid,
                isHost ? RtcRole.PUBLISHER : RtcRole.SUBSCRIBER, ttl, ttl);
  return { appId: APP_ID.value(), channel: classId, uid, token, expiresInSec: ttl };
});
```

13.4 Android side of the call

```
class AiRepositoryImpl @Inject constructor(private val functions: FirebaseFunctions) : AiRepository {
    override suspend fun ask(prompt: String): Result<String> = runCatching {
        val res = functions.getHttpsCallable("aiChat").call(mapOf("prompt" to prompt)).await()
        (res.data as Map<*, *>)["text"] as String
    }
}
// No key, URL or provider name is compiled into the app beyond the callable function name.
```

13.5 Rotation, limits and kill switches

- Rotate a provider key by adding a new Secret Manager version and redeploying the function; the APK is unaffected.
- Set spend limits and alerts in each provider console and a Google Cloud budget alert for the project.
- The AI kill switch (F352) is a Remote Config flag that the function also checks server-side; disabling it stops provider calls instantly.
- Never log prompts that contain personal data; log only sizes, latency and error class.

14. Security Architecture

Security is layered: a failure in one control must not defeat the whole system. The reference visuals below frame the same idea from two angles - a practical Android checklist and a levelled model that adds resilience against reverse engineering and tampering on top of standard and defence-in-depth controls.



Figure 14.1 - Layered security approach in Android: disable cleartext, certificate pinning, encrypted storage, encrypted database, R8 obfuscation, root detection (reference visual).

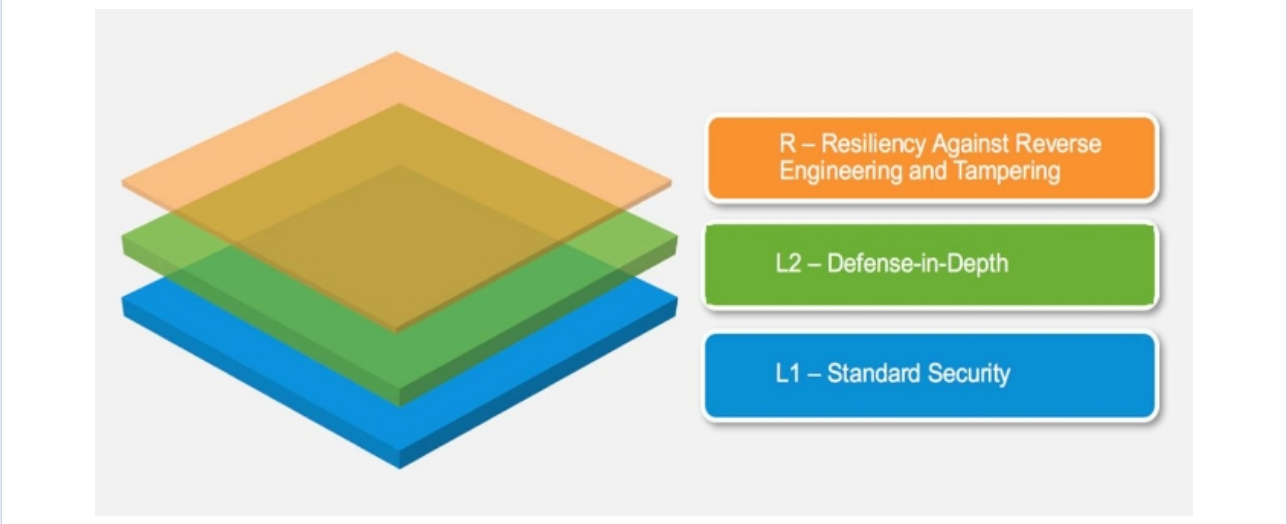


Figure 14.2 - Security levels: L1 standard security, L2 defence-in-depth, R resilience against reverse engineering and tampering (reference visual).

Level	Meaning for this app	Examples
L1 Standard	Baseline every release must meet	TLS only, secure storage, rules, input validation, no secrets in code
L2 Defence-in-depth	Controls that limit damage if L1 fails	MFA, App Check, rate limits, four-eyes approvals, audit log, encrypted backups
R Resilience	Raise the cost of reverse engineering and tampering	R8, integrity checks, root signals, FLAG_SECURE, tamper detection, server-side verdicts

14.1 Where the 50 controls apply

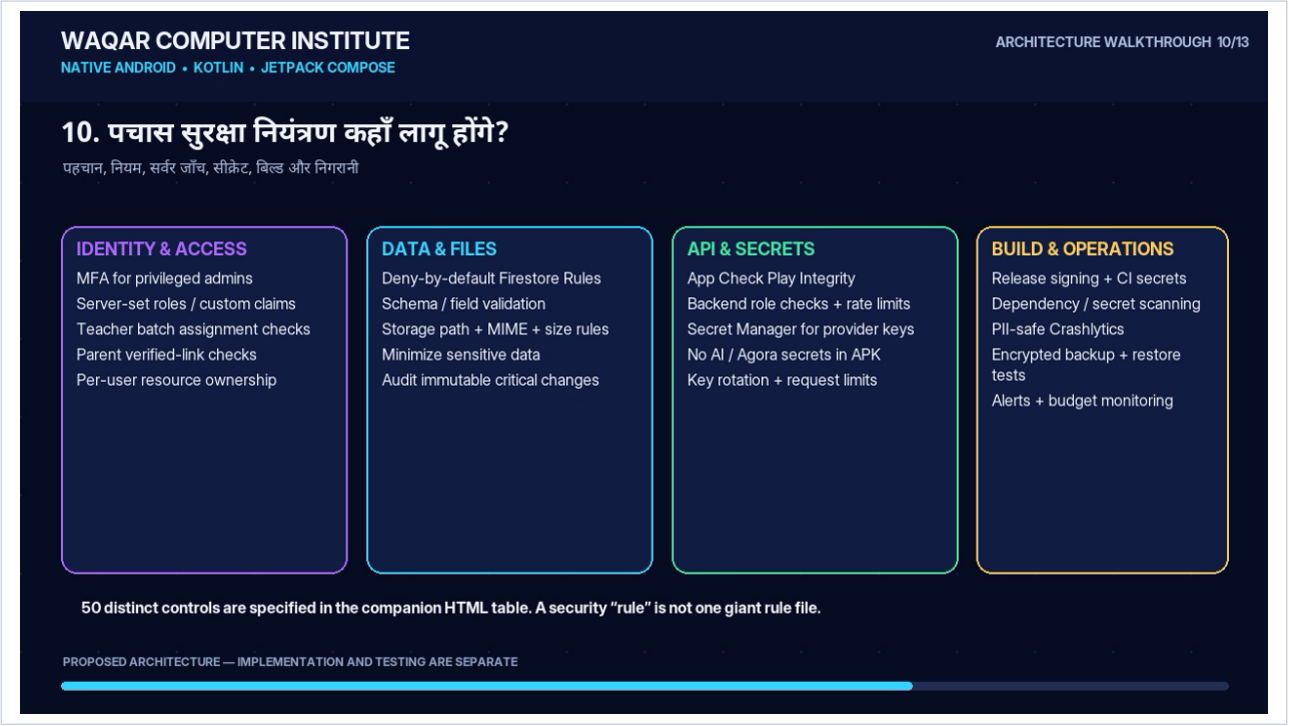


Figure 14.3 - Security controls grouped by identity, data, API and secrets, build and operations (video slide 10).

The complete table of 50 distinct controls is Appendix C. They are grouped into six areas: identity and access (10), data protection (10), network and transport (8), client hardening (10), secrets and backend (6) and build and operations (6).

14.2 Threat model summary

Threat	Example	Primary controls
Stolen or leaked secret	Provider key extracted from APK or repo	No keys in client (C39), Secret Manager, secret scanning, rotation
Account takeover	Guessed admin password	MFA, lockout, re-auth, session revocation, alerts
Privilege escalation	Teacher reads other batches	Server-set claims, rules scoped by batch, audit
Data scraping	Bulk download of student list	Deny-by-default rules, paging limits, rate limits, App Check
Modified APK	Repackaged app with hooks	Play Integrity, signature check, R8, server-side verdicts
Payment fraud	Fake webhook, replayed payment	Signature verification, idempotency, server-only fee writes
Insider mistake	Accidental mass delete	Soft delete, four-eyes, backups, delete protection
Supply chain	Malicious dependency	Pinned versions, dependency review, Dependabot, CI scanning

14.3 Network and manifest hardening

```
<!-- res/xml/network_security_config.xml -->
<network-security-config>
  <base-config cleartextTrafficPermitted="false">
    <trust-anchors><certificates src="system"/></trust-anchors>
  </base-config>
  <!-- Pin only first-party HTTPS endpoints you control. Firebase/Google endpoints are not pinned
       manually because Google rotates certificates. Always ship a backup pin. -->
</network-security-config>

<!-- AndroidManifest.xml (application) -->
android:usesCleartextTraffic="false"
android:allowBackup="false"
android:networkSecurityConfig="@xml/network_security_config"
android:debuggable="false"   <!-- release only; set by build type -->
```

14.4 Using encrypted storage correctly

Encrypted SharedPreferences, shown in the layered visual, is deprecated in current AndroidX Security guidance. The same protection is achieved with DataStore plus a Tink AEAD whose key is held in the Android Keystore. Only low-risk preferences and short-lived caches are stored; passwords and provider secrets are never stored on the device.

15. Libraries and Code Organisation

Fifty libraries are curated in Appendix B. They are selected for a reason, grouped by area, and added only when a feature needs them, so the APK stays small and the dependency surface stays reviewable.



Figure 15.1 - Where the fifty libraries fit: UI and navigation, app core and data, cloud and security, test and quality (video slide 8).

15.1 Version policy

- Use a Gradle version catalog (`libs.versions.toml`) so versions are declared once.
- Use the Compose BOM and Firebase BoM so related artifacts stay compatible.
- Pin exact versions; let Dependabot propose updates as pull requests that run the full CI.
- Check each version at implementation time against the official release notes. This document intentionally does not freeze version numbers.
- A library without recent maintenance (especially liquid-glass and 3D helpers) is wrapped behind an interface in `:core:ui` so it can be swapped.

15.2 Which Kotlin files do what



Figure 15.2 - Android core, feature module and cloud backend/rules file layout (video slide 9). The paths are proposed and are matched to the real repository during implementation.

```
repo-root/
  app/                                  // Android application module
  core/ui  core/domain  core/data  core/network  core/security
  feature/auth  feature/dashboard  feature/students  feature/attendance  feature/fees  feature/inbox
  feature/academics/...  feature/operations/...  feature/system/...
  functions/src/                        // Cloud Functions (TypeScript)
    index.ts  students/  fees/  ai/  live/  notifications/  jobs/  audit/
  firebase/firestore.rules  firebase/storage.rules  firebase/firestore.indexes.json  firebase.json
  scripts/bootstrap-admin.ts
  gradle/libs.versions.toml
  .github/workflows/android-ci.yml  android-release.yml  functions-deploy.yml  secret-scan.yml
```

16. CI/CD: GitHub Actions to a Signed Native APK

Every commit is checked; every merge to main can produce signed release artifacts. The ASCII flow supplied with the brief is the heart of it: build and test, then build release, then distribute to Firebase testers and the Play internal track.

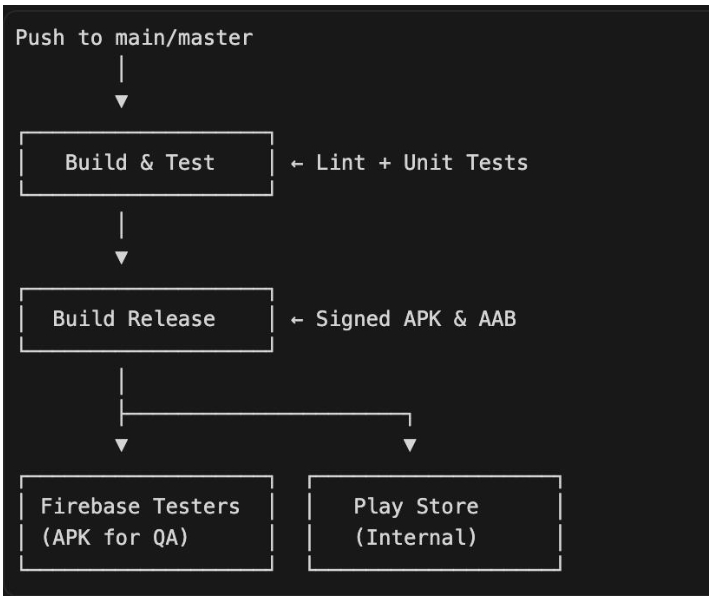


Figure 16.1 - Push to main: build and test (lint + unit tests), build release (signed APK and AAB), then Firebase Testers (APK) and Play Store internal (reference flow).

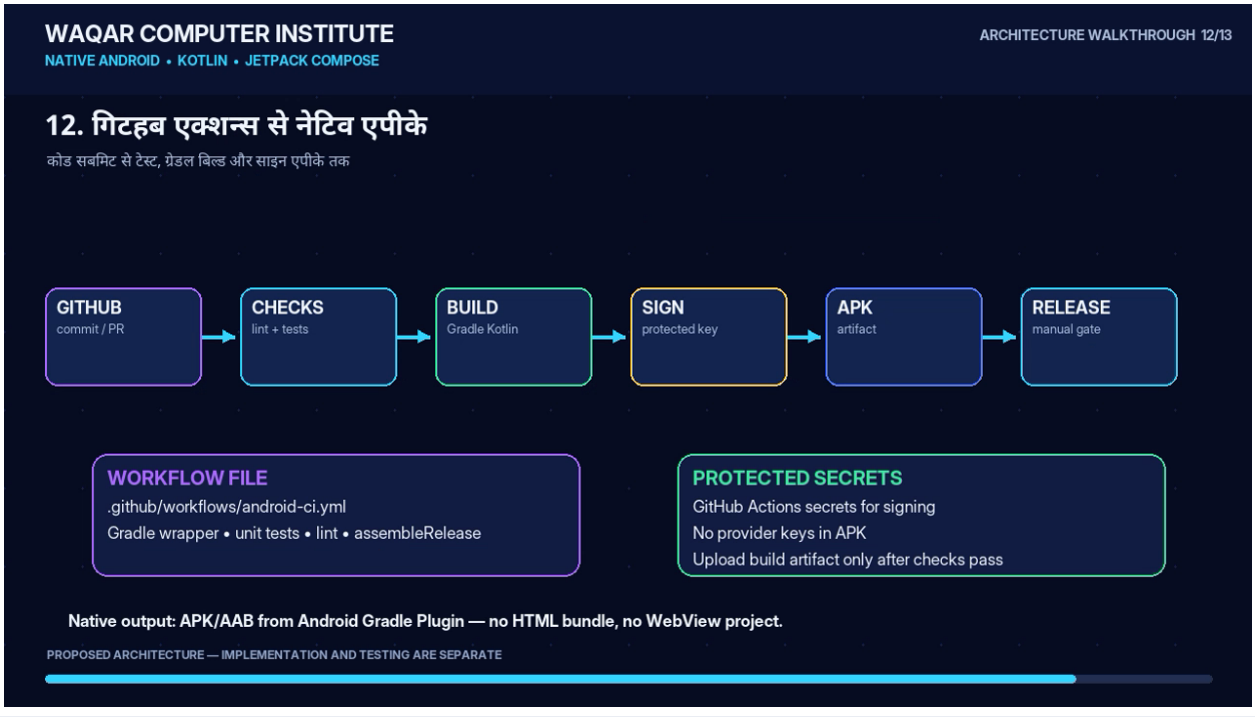


Figure 16.2 - GitHub commit, checks, Gradle build, signing with a protected key, APK artifact and manual release gate (video slide 12).



Figure 16.3 - Development to GitHub Actions (checkout, environment setup, lint and test, build, deploy) to production APK and AAB (reference diagram).

The fuller pipeline diagram below adds quality gates, security scanning, staging, staged production rollout, monitoring and a feedback loop. This blueprint adopts it, with the success metrics treated as targets to measure rather than promises.



Figure 16.4 - Android CI/CD pipeline: developer, source control, build, test, quality gate, security, package, staging, production, monitoring and feedback loop (reference diagram).

16.1 Required GitHub secrets

Secret name	Content	Used by
KEYSTORE_BASE64	Release keystore encoded with base64	android-release.yml
KEYSTORE_PASSWORD, KEY_ALIAS, KEY_PASSWORD	Signing credentials	android-release.yml
GOOGLE_SERVICES_JSON_BASE64	google-services.json (project configuration; restrict the API key to package + SHA-256)	Both Android workflows
FIREBASE_APP_ID	Firebase Android app ID	Distribution step
FIREBASE_SERVICE_ACCOUNT_JSON	Narrow-role service account for App Distribution (prefer Workload Identity Federation if available)	Distribution step
GCP deploy identity	Service account or federated identity limited to deploy functions and rules	functions-deploy.yml

No Gemini, OpenAI or Agora value appears in this list on purpose. They never enter GitHub.

16.2 Pull-request workflow: .github/workflows/android-ci.yml

```
name: android-ci
on:
  pull_request:
  push:
    branches: [main]
permissions:
  contents: read
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true
jobs:
  verify:
    runs-on: ubuntu-latest
    timeout-minutes: 40
    steps:
      - uses: actions/checkout@v4
      - name: Guard - native only (no WebView, no web bundle)
        run: |
          if grep -R --include=*.kt --include=*.java --include=*.xml -E "android.webkit.WebView|<WebView" app core feature; then
            echo "WebView usage is forbidden"; exit 1; fi
          if [ -d app/src/main/assets/www ] || [ -f app/src/main/assets/index.html ]; then
            echo "Web bundle found"; exit 1; fi
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '17' }
      - uses: gradle/actions/setup-gradle@v4
      - name: Restore Firebase config
        run: echo "${{ secrets.GOOGLE_SERVICES_JSON_BASE64 }}" | base64 -d > app/google-services.json
      - name: Lint, static analysis, unit tests
        run: ./gradlew ktlintCheck detekt lintRelease testReleaseUnitTest --stacktrace
      - name: Assemble release (unsigned) to prove it builds
        run: ./gradlew assembleRelease
      - uses: actions/upload-artifact@v4
        if: always()
        with: { name: reports, path: "**/build/reports/**" }
```

16.3 Release workflow: .github/workflows/android-release.yml

```
name: android-release
on:
  push:
    branches: [main]
  workflow_dispatch:
permissions:
  contents: read
jobs:
  release:
    runs-on: ubuntu-latest
    environment: production            # protected: requires manual approval
    timeout-minutes: 60
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '17' }
      - uses: gradle/actions/setup-gradle@v4
      - name: Decode secrets
        run: |
          echo "${{ secrets.GOOGLE_SERVICES_JSON_BASE64 }}" | base64 -d > app/google-services.json
          echo "${{ secrets.KEYSTORE_BASE64 }}" | base64 -d > "$RUNNER_TEMP/release.jks"
      - name: Test
        run: ./gradlew testReleaseUnitTest lintRelease
      - name: Build signed APK and AAB
        env:
          KEYSTORE_PATH: ${{ runner.temp }}/release.jks
          KEYSTORE_PASSWORD: ${{ secrets.KEYSTORE_PASSWORD }}
          KEY_ALIAS: ${{ secrets.KEY_ALIAS }}
          KEY_PASSWORD: ${{ secrets.KEY_PASSWORD }}
          VERSION_CODE: ${{ github.run_number }}
        run: ./gradlew assembleRelease bundleRelease
      - name: Remove keystore
        if: always()
        run: rm -f "$RUNNER_TEMP/release.jks"
      - uses: actions/upload-artifact@v4
        with:
          name: waqar-institute-release
          path: |
            app/build/outputs/apk/release/*.apk
            app/build/outputs/bundle/release/*.aab
            app/build/outputs/mapping/release/mapping.txt
      - name: Distribute APK to testers
        uses: wzieba/Firebase-Distribution-Github-Action@v1
        with:
```

```
          appId: ${{ secrets.FIREBASE_APP_ID }}
          serviceCredentialsFileContent: ${{ secrets.FIREBASE_SERVICE_ACCOUNT_JSON }}
          groups: qa
          file: app/build/outputs/apk/release/app-release.apk
```

16.4 Signing configuration in app/build.gradle.kts

```
android {
  signingConfigs {
    create("release") {
      val path = System.getenv("KEYSTORE_PATH")
      if (path != null) {
        storeFile = file(path)
        storePassword = System.getenv("KEYSTORE_PASSWORD")
        keyAlias = System.getenv("KEY_ALIAS")
        keyPassword = System.getenv("KEY_PASSWORD")
      }
    }
  }
  defaultConfig { versionCode = (System.getenv("VERSION_CODE") ?: "1").toInt() }
  buildTypes {
    release {
      isMinifyEnabled = true
      isShrinkResources = true
      proguardFiles(getDefaultProguardFile("proguard-android-optimize.txt"), "proguard-rules.pro")
      signingConfig = signingConfigs.getByName("release")
    }
  }
}
```

16.5 Supporting workflows

Workflow	Trigger	Job
secret-scan.yml	Push, pull request	Gitleaks scan of history and diff; fail on any finding
functions-deploy.yml	Push to main touching functions/ or firebase/	Lint, unit tests, deploy functions, rules and indexes to staging then production on approval
dependency-review	Pull request	Block vulnerable or disallowed-licence dependencies
nightly.yml	Schedule	Full instrumented tests on an emulator matrix, Macrobenchmark where available

16.6 Repository protection

- Branch protection on main: pull request required, status checks required, no force pushes.
- CODEOWNERS for security-sensitive paths: rules, functions, workflows, signing configuration.
- Environment "production" requires owner approval before the release job runs.
- Keep an offline encrypted backup of the keystore; losing it blocks updates of the published app. Use Play App Signing so Google holds the app signing key.

17. Testing, Quality and the "No Fake Data" Policy

A function is complete only when it works against the real Firebase project. The test strategy has four layers, each automated in CI where possible.

Layer	Scope	Tools	Gate
Unit	Use cases, validators, mappers, ViewModels	JUnit, Truth, MockK, Turbine	Every pull request
Rules and functions	Firestore and Storage rules, callable functions	Firebase Emulator Suite, rules unit-testing library, function unit tests	Every pull request touching firebase/ or functions/
UI	Compose screens and critical flows (login, add student, mark attendance, record payment)	Compose UI Test, Espresso	Nightly and before release
Performance and soak	Startup, list scroll, 100k-record paging, live-class join	Macrobenchmark, Baseline Profiles, Performance Monitoring	Before each release candidate

17.1 No dummy data, no fake functions

- **Production** contains only real data entered by the administrator or imported from the institute's real records. There is no seeding script for production.
- **Empty states** are designed screens ("No students yet - add the first student"), not sample content.
- **Emulator tests** create temporary documents inside the local emulator; they never touch the cloud project and are discarded after each run.
- **Load testing** for 100,000 records, if required, runs in a separate staging Firebase project that is clearly labelled and deleted afterwards. If the owner prefers zero synthetic data anywhere, performance is validated progressively on the real project as records are entered and imported.
- **Definition of done** for each of the 450 functions: screen exists, use case and repository implemented, backend path (rules or function) implemented, tests pass, empty/error/offline states handled, and an audit event exists where the function changes sensitive data.

17.2 Release checklist

#	Check	Evidence
1	All CI checks green; no WebView or web bundle	CI report
2	Rules and indexes deployed and tested	Emulator test log, console
3	No secret in repository history	Gitleaks report
4	R8 mapping file stored privately	Release artifact
5	Crashlytics and Performance enabled in release	Console
6	Budget alerts active for Firebase, Gemini, OpenAI, Agora	Console screenshots
7	Backup export completed and a restore drill passed this quarter	Backup centre log
8	Play Integrity and App Check enforced	Console
9	Accessibility pass (font scale, contrast, screen reader)	QA checklist
10	Owner sign-off on staged rollout	Approval in GitHub environment

18. Delivery Roadmap

Delivery is sequenced so that a secure, usable core exists early and each later phase adds modules without reworking the foundation. Durations depend on team size and are deliberately not stated here; estimate them after the team is confirmed.

Phase	Scope	Modules / functions	Exit criteria
0 Foundation	Repository, Gradle modules, design system skeleton, Firebase project, rules baseline, CI pipelines, bootstrap admin	Part of M01, M24	Signed APK builds in CI; admin can sign in with MFA; rules deployed
1 Core operations	Students, courses, batches, attendance, basic fees, dashboard, notifications	M02, M03, M05, M06, M07, M08, M16	Daily institute operations run in the app
2 Admissions and finance	Enquiries, admissions, receipts, accounts, payroll, guardians	M04, M09, M14, M15	Admission to receipt to ledger works end to end
3 Academics	Exams, material, assignments, certificates, reports	M10, M11, M12, M19	Results published; certificates verifiable
4 Live and AI	Agora live classes and AI tutor through functions	M17, M18	Token and AI proxies live; quotas enforced; kill switch tested
5 Operations and hardening	Inventory, support, security centre, backup tools, full settings, performance tuning	M13, M20, M21, M22, M23, M24	Restore drill passed; security review passed; staged rollout
6 Launch	Play Console internal, closed, then production rollout; monitoring	All	Crash-free target met on staged users

18.1 Team roles

Role	Responsibility
Android lead	Architecture, Compose design system, module boundaries, code review
Android developers	Feature modules, tests, performance
Backend developer	Cloud Functions, rules, indexes, jobs, integrations
UI/UX designer	Design system, motion, accessibility, asset pipeline
QA engineer	Test plans, device matrix, release checklist
DevOps / security	CI/CD, secrets, monitoring, security review
Owner (Waqar)	Priorities, approvals, data ownership, key custody

19. Risks, Open Decisions and Final Verdict

The video closes with the key conclusion reproduced below: the architecture is a blueprint, not a finished app. Each API, rule, feature, build and load test must be verified in the real repository.



Figure 19.1 - Final verdict: one student save, third-party calls, ship safely (video slide 13).

19.1 Risks

Risk	Impact	Mitigation
Scope: 450 functions with premium UI	Schedule and cost growth	Phased roadmap; function IDs allow scope control; MVP per phase
Visual effects on low-end phones	Jank, battery drain	Capability tiers, reduce-motion, benchmarks in CI
Firestore cost at scale	Unexpected bills	Paging, aggregation queries, counters, budget alerts
Third-party API cost and abuse	AI or live-class overspend	Per-user quotas, provider budgets, kill switch
Lost signing key	Cannot update the published app	Play App Signing, offline encrypted keystore backup
Liquid-glass or 3D library abandoned	Rework	Wrapper interfaces in :core:ui
Personal data of minors and parents	Legal and privacy exposure	Data minimisation, consent records, retention rules, access logs
Single-owner dependency	Operational risk	Second admin with MFA; documented recovery procedure

19.2 Decisions needed from the owner

- Firebase region and Blaze plan approval, including budget limits.
- Payment gateway choice (the design assumes a gateway with signed webhooks).
- Whether guardians sign in with phone OTP only, or also with a password.
- Branches: single institute or multi-branch (affects F372 and the data model).
- Retention periods for student, fee and audit records.
- Whether any synthetic data is acceptable in a separate staging project for load tests (section 17.1).
- Language support at launch (English only, or English + Hindi UI strings).

19.3 Final verdict

The architecture satisfies the brief: a native Kotlin Android app, 450 defined functions with explicit menu and tab placement, a Firestore model for 100,000 students, in-app credential management stored securely in the cloud, a premium tiered UI system, fifty libraries, fifty security controls, server-only handling of Gemini, OpenAI and Agora secrets, and a GitHub Actions pipeline that produces a signed APK and AAB. The next step is Phase 0 (Chapter 18).

Appendix A. The 450 Functions

Each function has a unique ID. **Screen** rows show the navigation placement (TAB, Drawer path, or Profile menu). All other rows show the screen that hosts them. The roles line under each module heading lists who may use the module; the real permission is always enforced by Firestore Rules or Cloud Functions, not only by the UI.

M01 Authentication & Session (18 functions)

Navigation: Pre-login flow + Profile menu (top-right avatar) **Roles:** All roles

ID	Function	Type	Appears in
F001	Login screen (username + password)	Screen	Pre-login flow + Profile menu (top-right avatar)
F002	Sign in with username alias	Action	In screen: Login screen (username + password)
F003	Input validation and lockout counter	Action	In screen: Login screen (username + password)
F004	Biometric unlock on app resume	Action	In screen: Login screen (username + password)
F005	Remember trusted device	Action	In screen: Login screen (username + password)
F006	Forgot password via verified email	Action	In screen: Login screen (username + password)
F007	Change password (re-authentication required)	Action	In screen: Login screen (username + password)
F008	Change username (server callable)	Action	In screen: Login screen (username + password)
F009	Enrol second factor (TOTP)	Action	In screen: Login screen (username + password)
F010	Verify second-factor challenge	Action	In screen: Login screen (username + password)
F011	Active sessions list	Screen	Pre-login flow + Profile menu (top-right avatar)
F012	Revoke single session	Action	In screen: Active sessions list
F013	Sign out all devices	Action	In screen: Active sessions list
F014	Auto-logout on idle	Background job	In screen: Active sessions list
F015	Token refresh and claim sync	Background job	In screen: Active sessions list
F016	Force re-login after role change	Background job	In screen: Active sessions list
F017	Login attempt history	List / filter	In screen: Active sessions list
F018	Session expiry banner	Action	In screen: Active sessions list

M02 Admin Dashboard (16 functions)

Navigation: TAB 1: Home **Roles:** Admin

ID	Function	Type	Appears in
F019	Home dashboard	Screen	TAB 1: Home
F020	Student count tile (aggregate query)	List / filter	In screen: Home dashboard
F021	New admissions this month tile	List / filter	In screen: Home dashboard
F022	Today's attendance tile	List / filter	In screen: Home dashboard
F023	Fee collected today tile	List / filter	In screen: Home dashboard
F024	Pending dues tile	List / filter	In screen: Home dashboard
F025	Upcoming batches tile	List / filter	In screen: Home dashboard
F026	Upcoming exams tile	List / filter	In screen: Home dashboard
F027	Birthdays and anniversaries card	List / filter	In screen: Home dashboard
F028	Quick actions bar	Action	In screen: Home dashboard
F029	Pending approvals queue	List / filter	In screen: Home dashboard
F030	Recent activity feed (audit tail)	List / filter	In screen: Home dashboard
F031	Revenue trend chart	Report / export	In screen: Home dashboard
F032	Enrolment trend chart	Report / export	In screen: Home dashboard
F033	Pull-to-refresh and live-listener toggle	Action	In screen: Home dashboard
F034	Dashboard widget layout preference	Configuration	In screen: Home dashboard

M03 Student Management (30 functions)

Navigation: TAB 2: Students **Roles:** Admin (write); Teacher (read, own batches)

ID	Function	Type	Appears in
F035	Students list (paged)	Screen	TAB 2: Students
F036	Search by name prefix	List / filter	In screen: Students list (paged)
F037	Search by phone or enrolment number	List / filter	In screen: Students list (paged)
F038	Filter by course	List / filter	In screen: Students list (paged)
F039	Filter by batch	List / filter	In screen: Students list (paged)
F040	Filter by status	List / filter	In screen: Students list (paged)
F041	Filter by fee-due state	List / filter	In screen: Students list (paged)
F042	Sort options	List / filter	In screen: Students list (paged)
F043	Add student form	Screen	TAB 2: Students
F044	Duplicate phone/email check	Action	In screen: Add student form
F045	Auto-generate enrolment number (transaction)	Action	In screen: Add student form
F046	Upload student photo	Action	In screen: Add student form
F047	Upload ID proof	Action	In screen: Add student form
F048	Student profile detail	Screen	TAB 2: Students
F049	Edit student	Action	In screen: Student profile detail
F050	Archive student	Action	In screen: Student profile detail
F051	Restore archived student	Action	In screen: Student profile detail
F052	Soft-delete student with audit	Action	In screen: Student profile detail
F053	Assign student to course	Action	In screen: Student profile detail
F054	Assign student to batch	Action	In screen: Student profile detail
F055	Transfer student between batches	Action	In screen: Student profile detail
F056	Link guardian	Action	In screen: Student profile detail
F057	Student notes and remarks	Action	In screen: Student profile detail
F058	Student document vault	Screen	TAB 2: Students
F059	Generate student ID card	Action	In screen: Student document vault
F060	Generate student QR code	Action	In screen: Student document vault
F061	Bulk import (CSV via Cloud Function)	Action	In screen: Student document vault
F062	Bulk export (CSV/PDF)	Report / export	In screen: Student document vault
F063	Bulk status change	Action	In screen: Student document vault
F064	Student activity timeline	List / filter	In screen: Student document vault

M04 Admissions & Enquiries (20 functions)

Navigation: Drawer > People > Admissions **Roles:** Admin

ID	Function	Type	Appears in
F065	Enquiry list	Screen	Drawer > People > Admissions
F066	Add enquiry	Action	In screen: Enquiry list
F067	Enquiry source tracking	Configuration	In screen: Enquiry list
F068	Follow-up scheduler	Action	In screen: Enquiry list
F069	Follow-up reminder notification	Background job	In screen: Enquiry list
F070	Enquiry pipeline board	Screen	Drawer > People > Admissions
F071	Assign enquiry to counsellor	Action	In screen: Enquiry pipeline board
F072	Call log entry	Action	In screen: Enquiry pipeline board
F073	Convert enquiry to admission	Action	In screen: Enquiry pipeline board
F074	Admission form	Screen	Drawer > People > Admissions
F075	Upload admission documents	Action	In screen: Admission form

ID	Function	Type	Appears in
F076	Admission fee capture at entry	Action	In screen: Admission form
F077	Scholarship / discount apply	Action	In screen: Admission form
F078	Admission approval workflow	Action	In screen: Admission form
F079	Reject admission with reason	Action	In screen: Admission form
F080	Waitlist management	Screen	Drawer > People > Admissions
F081	Admission confirmation message	Action	In screen: Waitlist management
F082	Enquiry-to-admission funnel report	Report / export	In screen: Waitlist management
F083	Counsellor performance report	Report / export	In screen: Waitlist management
F084	Duplicate enquiry detection	Background job	In screen: Waitlist management

M05 Course Catalog (18 functions)

Navigation: Drawer > Academics > Courses **Roles:** Admin (write); Teacher (read)

ID	Function	Type	Appears in
F085	Courses list	Screen	Drawer > Academics > Courses
F086	Add course	Action	In screen: Courses list
F087	Edit course	Action	In screen: Courses list
F088	Archive course	Action	In screen: Courses list
F089	Course syllabus builder	Screen	Drawer > Academics > Courses
F090	Module and lesson ordering	Action	In screen: Course syllabus builder
F091	Course duration and schedule rules	Configuration	In screen: Course syllabus builder
F092	Course fee plan template	Configuration	In screen: Course syllabus builder
F093	Course prerequisites	Configuration	In screen: Course syllabus builder
F094	Course category management	Screen	Drawer > Academics > Courses
F095	Course eligibility rules	Configuration	In screen: Course category management
F096	Certificate template link	Configuration	In screen: Course category management
F097	Course demand heatmap	Report / export	In screen: Course category management
F098	Course capacity limit	Configuration	In screen: Course category management
F099	Course trainer mapping	Action	In screen: Course category management
F100	Course thumbnail upload	Action	In screen: Course category management
F101	Publish / unpublish course	Action	In screen: Course category management
F102	Course version history	List / filter	In screen: Course category management

M06 Batch & Timetable (22 functions)

Navigation: Drawer > Academics > Batches **Roles:** Admin (write); Teacher (own batches); Student/Parent (read own)

ID	Function	Type	Appears in
F103	Batches list	Screen	Drawer > Academics > Batches
F104	Create batch	Action	In screen: Batches list
F105	Edit batch	Action	In screen: Batches list
F106	Close batch	Action	In screen: Batches list
F107	Batch capacity monitor	List / filter	In screen: Batches list
F108	Batch student roster	List / filter	In screen: Batches list
F109	Add student to batch	Action	In screen: Batches list
F110	Remove student from batch	Action	In screen: Batches list
F111	Timetable calendar	Screen	Drawer > Academics > Batches
F112	Create timetable slot	Action	In screen: Timetable calendar
F113	Recurring slot rule	Action	In screen: Timetable calendar

ID	Function	Type	Appears in
F114	Room allocation	Action	In screen: Timetable calendar
F115	Trainer allocation	Action	In screen: Timetable calendar
F116	Clash detection (room/trainer)	Background job	In screen: Timetable calendar
F117	Reschedule class	Action	In screen: Timetable calendar
F118	Cancel class with notice	Action	In screen: Timetable calendar
F119	Holiday calendar	Screen	Drawer > Academics > Batches
F120	Add holiday	Action	In screen: Holiday calendar
F121	Substitute trainer assign	Action	In screen: Holiday calendar
F122	Timetable export (PDF)	Report / export	In screen: Holiday calendar
F123	Timetable share to students	Action	In screen: Holiday calendar
F124	Batch completion tracker	List / filter	In screen: Holiday calendar

M07 Attendance (20 functions)

Navigation: TAB 3: Attendance **Roles:** Admin; Teacher (own batches); Student/Parent (read own)

ID	Function	Type	Appears in
F125	Attendance home (today)	Screen	TAB 3: Attendance
F126	Mark student attendance	Action	In screen: Attendance home (today)
F127	Mark with QR scan	Action	In screen: Attendance home (today)
F128	Mark with roster tap	Action	In screen: Attendance home (today)
F129	Mark late / excused	Action	In screen: Attendance home (today)
F130	Bulk mark all present	Action	In screen: Attendance home (today)
F131	Edit attendance (reason + audit)	Action	In screen: Attendance home (today)
F132	Lock attendance after cut-off	Background job	In screen: Attendance home (today)
F133	Attendance by batch	List / filter	In screen: Attendance home (today)
F134	Attendance by student	List / filter	In screen: Attendance home (today)
F135	Attendance percentage calculation	Background job	In screen: Attendance home (today)
F136	Low-attendance alert	Background job	In screen: Attendance home (today)
F137	Parent absence notice	Background job	In screen: Attendance home (today)
F138	Staff attendance mark	Action	In screen: Attendance home (today)
F139	Staff attendance by QR / geo-fence	Action	In screen: Attendance home (today)
F140	Leave-linked attendance sync	Background job	In screen: Attendance home (today)
F141	Attendance monthly report	Report / export	In screen: Attendance home (today)
F142	Attendance heatmap	Report / export	In screen: Attendance home (today)
F143	Attendance export	Report / export	In screen: Attendance home (today)
F144	Offline attendance queue (WorkManager)	Background job	In screen: Attendance home (today)

M08 Fees & Payments (28 functions)

Navigation: TAB 4: Fees **Roles:** Admin (write); Parent/Student (read own)

ID	Function	Type	Appears in
F145	Fees home	Screen	TAB 4: Fees
F146	Fee structure per course	Configuration	In screen: Fees home
F147	Instalment plan builder	Action	In screen: Fees home
F148	Assign fee plan to student	Action	In screen: Fees home
F149	Record cash payment	Action	In screen: Fees home
F150	Record bank / UPI payment	Action	In screen: Fees home
F151	Online payment via gateway (server verified)	Action	In screen: Fees home
F152	Payment webhook verification	Background job	In screen: Fees home

ID	Function	Type	Appears in
F153	Generate receipt	Action	In screen: Fees home
F154	Share receipt PDF	Action	In screen: Fees home
F155	Fee due list	List / filter	In screen: Fees home
F156	Overdue list	List / filter	In screen: Fees home
F157	Scheduled due reminders	Background job	In screen: Fees home
F158	Apply discount	Action	In screen: Fees home
F159	Late-fee rule	Configuration	In screen: Fees home
F160	Refund request	Action	In screen: Fees home
F161	Approve refund	Action	In screen: Fees home
F162	Fee waiver	Action	In screen: Fees home
F163	Concession approval workflow	Action	In screen: Fees home
F164	Fee ledger per student	Screen	TAB 4: Fees
F165	Daily collection summary	Report / export	In screen: Fee ledger per student
F166	Collection by payment mode	Report / export	In screen: Fee ledger per student
F167	Collection by course	Report / export	In screen: Fee ledger per student
F168	Collection by staff	Report / export	In screen: Fee ledger per student
F169	Fee defaulters report	Report / export	In screen: Fee ledger per student
F170	Cheque / bounce tracking	Action	In screen: Fee ledger per student
F171	Fee audit trail	List / filter	In screen: Fee ledger per student
F172	Fee data export	Report / export	In screen: Fee ledger per student

M09 Accounts & Expenses (18 functions)

Navigation: Drawer > Finance > Accounts **Roles:** Admin

ID	Function	Type	Appears in
F173	Expense list	Screen	Drawer > Finance > Accounts
F174	Add expense	Action	In screen: Expense list
F175	Expense categories	Configuration	In screen: Expense list
F176	Attach bill photo	Action	In screen: Expense list
F177	Vendor directory	Screen	Drawer > Finance > Accounts
F178	Add vendor	Action	In screen: Vendor directory
F179	Vendor payment	Action	In screen: Vendor directory
F180	Recurring expense schedule	Action	In screen: Vendor directory
F181	Petty cash register	Screen	Drawer > Finance > Accounts
F182	Petty cash top-up	Action	In screen: Petty cash register
F183	Non-fee income entry	Action	In screen: Petty cash register
F184	Profit and loss report	Report / export	In screen: Petty cash register
F185	Monthly balance view	Report / export	In screen: Petty cash register
F186	Budget plan	Configuration	In screen: Petty cash register
F187	Budget vs actual	Report / export	In screen: Petty cash register
F188	Tax / GST summary report	Report / export	In screen: Petty cash register
F189	Expense approval	Action	In screen: Petty cash register
F190	Financial year close	Action	In screen: Petty cash register

M10 Examinations & Results (24 functions)

Navigation: Drawer > Academics > Exams **Roles:** Admin; Teacher (own batches); Student/Parent (read own)

ID	Function	Type	Appears in
F191	Exams list	Screen	Drawer > Academics > Exams

ID	Function	Type	Appears in
F192	Create exam	Action	In screen: Exams list
F193	Exam schedule	Action	In screen: Exams list
F194	Question bank	Screen	Drawer > Academics > Exams
F195	Add question	Action	In screen: Question bank
F196	Import questions (CSV)	Action	In screen: Question bank
F197	Paper builder	Action	In screen: Question bank
F198	Online test attempt (student)	Screen	Drawer > Academics > Exams
F199	Timer and autosave of attempt	Background job	In screen: Online test attempt (student)
F200	Auto-evaluate objective answers (server)	Background job	In screen: Online test attempt (student)
F201	Manual marking screen	Screen	Drawer > Academics > Exams
F202	Marks entry grid	Action	In screen: Manual marking screen
F203	Grace-marks rule	Configuration	In screen: Manual marking screen
F204	Grade scale config	Configuration	In screen: Manual marking screen
F205	Result calculation	Background job	In screen: Manual marking screen
F206	Result publish / hold	Action	In screen: Manual marking screen
F207	Re-evaluation request	Action	In screen: Manual marking screen
F208	Rank list	Report / export	In screen: Manual marking screen
F209	Subject-wise analysis	Report / export	In screen: Manual marking screen
F210	Student result card PDF	Report / export	In screen: Manual marking screen
F211	Bulk result export	Report / export	In screen: Manual marking screen
F212	Hall ticket generate	Action	In screen: Manual marking screen
F213	Invigilator assignment	Action	In screen: Manual marking screen
F214	Exam attendance	Action	In screen: Manual marking screen

M11 Assignments & Study Material (18 functions)

Navigation: Drawer > Academics > Material **Roles:** Admin; Teacher (write); Student (read assigned)

ID	Function	Type	Appears in
F215	Material library	Screen	Drawer > Academics > Material
F216	Upload PDF / video / notes	Action	In screen: Material library
F217	Material categories	Configuration	In screen: Material library
F218	Assign material to batch	Action	In screen: Material library
F219	Visibility and expiry rules	Configuration	In screen: Material library
F220	Download tracking	List / filter	In screen: Material library
F221	Assignments list	Screen	Drawer > Academics > Material
F222	Create assignment	Action	In screen: Assignments list
F223	Due-date and late policy	Configuration	In screen: Assignments list
F224	Student submission upload	Action	In screen: Assignments list
F225	Grade submission	Action	In screen: Assignments list
F226	Feedback comments	Action	In screen: Assignments list
F227	Similarity flag on submissions (server)	Background job	In screen: Assignments list
F228	Allow resubmission	Action	In screen: Assignments list
F229	Submission status board	List / filter	In screen: Assignments list
F230	Attach recorded lecture	Action	In screen: Assignments list
F231	Material search	List / filter	In screen: Assignments list
F232	Storage usage per batch	Report / export	In screen: Assignments list

M12 Certificates (14 functions)

Navigation: Drawer > Academics > Certificates **Roles:** Admin; Student (read own)

ID	Function	Type	Appears in
F233	Certificates home	Screen	Drawer > Academics > Certificates
F234	Certificate templates	Screen	Drawer > Academics > Certificates
F235	Template field editor	Action	In screen: Certificate templates
F236	Issue certificate	Action	In screen: Certificate templates
F237	Bulk issue to batch	Action	In screen: Certificate templates
F238	Certificate number sequence	Background job	In screen: Certificate templates
F239	QR verification code	Action	In screen: Certificate templates
F240	Public verify endpoint (callable)	Background job	In screen: Certificate templates
F241	Revoke certificate	Action	In screen: Certificate templates
F242	Reissue certificate	Action	In screen: Certificate templates
F243	Certificate PDF render	Action	In screen: Certificate templates
F244	Share certificate	Action	In screen: Certificate templates
F245	Eligibility check (attendance + results + fees)	Background job	In screen: Certificate templates
F246	Issued certificates register	Report / export	In screen: Certificate templates

M13 Teacher & Staff Management (22 functions)

Navigation: Drawer > People > Staff **Roles:** Admin

ID	Function	Type	Appears in
F247	Staff list	Screen	Drawer > People > Staff
F248	Add staff	Action	In screen: Staff list
F249	Edit staff	Action	In screen: Staff list
F250	Deactivate staff	Action	In screen: Staff list
F251	Staff profile	Screen	Drawer > People > Staff
F252	Role assignment (custom claim)	Action	In screen: Staff profile
F253	Permission matrix editor	Screen	Drawer > People > Staff
F254	Assign batches to teacher	Action	In screen: Permission matrix editor
F255	Teacher workload view	List / filter	In screen: Permission matrix editor
F256	Staff documents	Action	In screen: Permission matrix editor
F257	Staff photo	Action	In screen: Permission matrix editor
F258	Qualification records	Action	In screen: Permission matrix editor
F259	Onboarding checklist	Action	In screen: Permission matrix editor
F260	Offboarding checklist	Action	In screen: Permission matrix editor
F261	Staff directory search	List / filter	In screen: Permission matrix editor
F262	Staff performance review	Action	In screen: Permission matrix editor
F263	Student feedback on teacher view	List / filter	In screen: Permission matrix editor
F264	Teacher availability calendar	Screen	Drawer > People > Staff
F265	Invite staff (email link)	Action	In screen: Teacher availability calendar
F266	Admin-initiated staff password reset	Action	In screen: Teacher availability calendar
F267	Staff import	Action	In screen: Teacher availability calendar
F268	Staff export	Report / export	In screen: Teacher availability calendar

M14 Payroll & Leave (16 functions)

Navigation: Drawer > Finance > Payroll **Roles:** Admin; Staff (own payslip, leave)

ID	Function	Type	Appears in
F269	Payroll home	Screen	Drawer > Finance > Payroll
F270	Salary structure	Configuration	In screen: Payroll home
F271	Assign salary to staff	Action	In screen: Payroll home
F272	Monthly payroll run	Action	In screen: Payroll home
F273	Generate payslip	Action	In screen: Payroll home
F274	Share payslip	Action	In screen: Payroll home
F275	Deductions and advances	Action	In screen: Payroll home
F276	Leave types	Configuration	In screen: Payroll home
F277	Leave request (staff)	Action	In screen: Payroll home
F278	Leave approval	Action	In screen: Payroll home
F279	Leave balance view	List / filter	In screen: Payroll home
F280	Holiday-linked leave rule	Configuration	In screen: Payroll home
F281	Payroll register report	Report / export	In screen: Payroll home
F282	Payroll bank-file export	Report / export	In screen: Payroll home
F283	Overtime entry	Action	In screen: Payroll home
F284	Lock payroll month	Action	In screen: Payroll home

M15 Guardian (Parent) Linking (12 functions)

Navigation: Drawer > People > Guardians **Roles:** Admin; Parent (own profile)

ID	Function	Type	Appears in
F285	Guardian list	Screen	Drawer > People > Guardians
F286	Add guardian	Action	In screen: Guardian list
F287	Verify guardian phone (OTP)	Action	In screen: Guardian list
F288	Link guardian to student	Action	In screen: Guardian list
F289	Unlink guardian	Action	In screen: Guardian list
F290	Guardian visibility rules	Configuration	In screen: Guardian list
F291	Parent notification preferences	Configuration	In screen: Guardian list
F292	Parent fee-view access	Configuration	In screen: Guardian list
F293	Parent result-view access	Configuration	In screen: Guardian list
F294	Guardian message thread	Screen	Drawer > People > Guardians
F295	Guardian consent record	Action	In screen: Guardian message thread
F296	Guardian invite link	Action	In screen: Guardian message thread

M16 Notifications & Announcements (18 functions)

Navigation: TAB 5: Inbox **Roles:** Admin (send); Teacher (send to own batches); all (receive)

ID	Function	Type	Appears in
F297	Announcements list	Screen	TAB 5: Inbox
F298	Create announcement	Action	In screen: Announcements list
F299	Target by batch / course / role	Action	In screen: Announcements list
F300	Schedule announcement	Action	In screen: Announcements list
F301	Push notification send (FCM via function)	Action	In screen: Announcements list
F302	Notification templates	Configuration	In screen: Announcements list
F303	In-app inbox	Screen	TAB 5: Inbox
F304	Mark read / unread	Action	In screen: In-app inbox
F305	Delivery status tracking	List / filter	In screen: In-app inbox
F306	Topic subscription management	Background job	In screen: In-app inbox
F307	Birthday greeting automation	Background job	In screen: In-app inbox

ID	Function	Type	Appears in
F308	Fee reminder template	Configuration	In screen: In-app inbox
F309	SMS / WhatsApp hand-off link	Action	In screen: In-app inbox
F310	Emergency broadcast	Action	In screen: In-app inbox
F311	Quiet-hours rules	Configuration	In screen: In-app inbox
F312	Notification history	List / filter	In screen: In-app inbox
F313	Attachment to announcement	Action	In screen: In-app inbox
F314	Announcement reach analytics	Report / export	In screen: In-app inbox

M17 Live Classes (Agora) (20 functions)

Navigation: Drawer > Academics > Live Classes **Roles:** Admin; Teacher (host); Student (join assigned)

ID	Function	Type	Appears in
F315	Live classes home	Screen	Drawer > Academics > Live Classes
F316	Schedule live class	Action	In screen: Live classes home
F317	Start live class (teacher)	Action	In screen: Live classes home
F318	Join live class (student)	Action	In screen: Live classes home
F319	Server-minted Agora token	Background job	In screen: Live classes home
F320	Channel access check (role + batch)	Background job	In screen: Live classes home
F321	Camera / microphone permission flow	Action	In screen: Live classes home
F322	Mute / unmute participant	Action	In screen: Live classes home
F323	Raise hand	Action	In screen: Live classes home
F324	Screen share	Action	In screen: Live classes home
F325	In-class chat	Action	In screen: Live classes home
F326	Recording trigger	Action	In screen: Live classes home
F327	Recordings list	Screen	Drawer > Academics > Live Classes
F328	Auto-attendance from join log	Background job	In screen: Recordings list
F329	Remove participant	Action	In screen: Recordings list
F330	Network quality indicator	List / filter	In screen: Recordings list
F331	Low-bandwidth audio-only mode	Action	In screen: Recordings list
F332	Class end summary	Report / export	In screen: Recordings list
F333	Live class reminders	Background job	In screen: Recordings list
F334	Concurrent channel usage monitor	Report / export	In screen: Recordings list

M18 AI Tutor & Assistant (18 functions)

Navigation: Drawer > Academics > AI Hub **Roles:** Student/Teacher (tutor); Admin (assistant, config)

ID	Function	Type	Appears in
F335	AI hub	Screen	Drawer > Academics > AI Hub
F336	Student AI tutor chat	Screen	Drawer > Academics > AI Hub
F337	Ask with attachment (image / PDF)	Action	In screen: Student AI tutor chat
F338	Server proxy call (Gemini)	Background job	In screen: Student AI tutor chat
F339	Server proxy call (OpenAI)	Background job	In screen: Student AI tutor chat
F340	Provider routing and fallback	Background job	In screen: Student AI tutor chat
F341	Per-user quota and rate limits	Background job	In screen: Student AI tutor chat
F342	Prompt templates (admin)	Configuration	In screen: Student AI tutor chat
F343	Safety and content filter	Background job	In screen: Student AI tutor chat
F344	Conversation history	List / filter	In screen: Student AI tutor chat
F345	Delete conversation	Action	In screen: Student AI tutor chat
F346	Admin AI assistant (questions over reports)	Screen	Drawer > Academics > AI Hub

ID	Function	Type	Appears in
F347	AI-generated quiz from material	Action	In screen: Admin AI assistant (questions over reports)
F348	AI-generated lesson summary	Action	In screen: Admin AI assistant (questions over reports)
F349	AI answer feedback	Action	In screen: Admin AI assistant (questions over reports)
F350	Usage and cost dashboard	Report / export	In screen: Admin AI assistant (questions over reports)
F351	Provider key-rotation trigger	Configuration	In screen: Admin AI assistant (questions over reports)
F352	AI kill switch	Configuration	In screen: Admin AI assistant (questions over reports)

M19 Reports & Analytics (22 functions)

Navigation: Drawer > Insights > Reports **Roles:** Admin; Teacher (own batches, limited)

ID	Function	Type	Appears in
F353	Reports home	Screen	Drawer > Insights > Reports
F354	Student strength report	Report / export	In screen: Reports home
F355	Admission report	Report / export	In screen: Reports home
F356	Course-wise enrolment	Report / export	In screen: Reports home
F357	Batch-wise report	Report / export	In screen: Reports home
F358	Attendance summary report	Report / export	In screen: Reports home
F359	Fee collection report	Report / export	In screen: Reports home
F360	Dues ageing report	Report / export	In screen: Reports home
F361	Exam performance report	Report / export	In screen: Reports home
F362	Teacher workload report	Report / export	In screen: Reports home
F363	Dropout-risk list	Report / export	In screen: Reports home
F364	Custom report builder	Screen	Drawer > Insights > Reports
F365	Save report preset	Action	In screen: Custom report builder
F366	Schedule report email	Background job	In screen: Custom report builder
F367	Export to PDF	Report / export	In screen: Custom report builder
F368	Export to Excel / CSV	Report / export	In screen: Custom report builder
F369	Share report	Action	In screen: Custom report builder
F370	Chart drill-down	Action	In screen: Custom report builder
F371	Date-range comparison	Action	In screen: Custom report builder
F372	Branch comparison (multi-branch)	Report / export	In screen: Custom report builder
F373	Data freshness indicator	List / filter	In screen: Custom report builder
F374	BigQuery export toggle	Configuration	In screen: Custom report builder

M20 Inventory, Lab & Library (14 functions)

Navigation: Drawer > Operations > Inventory **Roles:** Admin

ID	Function	Type	Appears in
F375	Inventory home	Screen	Drawer > Operations > Inventory
F376	Add asset	Action	In screen: Inventory home
F377	Asset categories	Configuration	In screen: Inventory home
F378	Lab computer register	Screen	Drawer > Operations > Inventory
F379	Assign asset to lab seat	Action	In screen: Lab computer register
F380	Maintenance log	Action	In screen: Lab computer register
F381	Library books list	Screen	Drawer > Operations > Inventory

ID	Function	Type	Appears in
F382	Issue book	Action	In screen: Library books list
F383	Return book	Action	In screen: Library books list
F384	Fine calculation	Background job	In screen: Library books list
F385	Stock count	Action	In screen: Library books list
F386	Low-stock alert	Background job	In screen: Library books list
F387	Asset QR labels	Action	In screen: Library books list
F388	Asset report	Report / export	In screen: Library books list

M21 Support & Feedback (12 functions)

Navigation: Drawer > Operations > Support **Roles:** Admin; Teacher; Student/Parent (create, read own)

ID	Function	Type	Appears in
F389	Support tickets list	Screen	Drawer > Operations > Support
F390	Create ticket	Action	In screen: Support tickets list
F391	Assign ticket	Action	In screen: Support tickets list
F392	Update ticket status	Action	In screen: Support tickets list
F393	Ticket chat thread	Screen	Drawer > Operations > Support
F394	Attach screenshot	Action	In screen: Ticket chat thread
F395	Student feedback form	Screen	Drawer > Operations > Support
F396	Course feedback survey	Action	In screen: Student feedback form
F397	Feedback analytics	Report / export	In screen: Student feedback form
F398	Complaint escalation	Action	In screen: Student feedback form
F399	Canned responses	Configuration	In screen: Student feedback form
F400	Ticket SLA timers	Background job	In screen: Student feedback form

M22 Security & Audit (20 functions)

Navigation: Drawer > System > Security **Roles:** Admin (owner) only

ID	Function	Type	Appears in
F401	Security centre	Screen	Drawer > System > Security
F402	Audit log viewer	Screen	Drawer > System > Security
F403	Audit filter and search	List / filter	In screen: Audit log viewer
F404	Audit export	Report / export	In screen: Audit log viewer
F405	Role and claim viewer	List / filter	In screen: Audit log viewer
F406	Suspicious sign-in alerts	Background job	In screen: Audit log viewer
F407	Device integrity status (Play Integrity)	List / filter	In screen: Audit log viewer
F408	App Check enforcement status	List / filter	In screen: Audit log viewer
F409	Lock account	Action	In screen: Audit log viewer
F410	Unlock account	Action	In screen: Audit log viewer
F411	Force password reset	Action	In screen: Audit log viewer
F412	Device / network allow rules (optional)	Configuration	In screen: Audit log viewer
F413	Sensitive-data access log	List / filter	In screen: Audit log viewer
F414	Consent and privacy requests	Screen	Drawer > System > Security
F415	Data erasure request handling	Action	In screen: Consent and privacy requests
F416	Session policy config	Configuration	In screen: Consent and privacy requests
F417	Security-rules deployment status	List / filter	In screen: Consent and privacy requests
F418	Secret-rotation reminders	Background job	In screen: Consent and privacy requests
F419	Four-eyes approval for critical changes	Action	In screen: Consent and privacy requests
F420	Security incident log	Screen	Drawer > System > Security

M23 Backup & Data Tools (14 functions)

Navigation: Drawer > System > Backup **Roles:** Admin (owner) only

ID	Function	Type	Appears in
F421	Backup centre	Screen	Drawer > System > Backup
F422	Scheduled Firestore export	Background job	In screen: Backup centre
F423	Manual export now	Action	In screen: Backup centre
F424	Backup status and history	List / filter	In screen: Backup centre
F425	Restore drill checklist	Screen	Drawer > System > Backup
F426	Point-in-time recovery information	List / filter	In screen: Restore drill checklist
F427	Storage backup	Background job	In screen: Restore drill checklist
F428	Data integrity checker	Background job	In screen: Restore drill checklist
F429	Orphan record cleaner	Background job	In screen: Restore drill checklist
F430	Import validator	Action	In screen: Restore drill checklist
F431	Data retention rules	Configuration	In screen: Restore drill checklist
F432	Archive old academic years	Action	In screen: Restore drill checklist
F433	Export CSV bundle	Report / export	In screen: Restore drill checklist
F434	Delete-protection status	List / filter	In screen: Restore drill checklist

M24 Settings & Institute Profile (16 functions)

Navigation: Drawer > System > Settings **Roles:** Admin (owner)

ID	Function	Type	Appears in
F435	Settings home	Screen	Drawer > System > Settings
F436	Institute profile (name, logo, address)	Screen	Drawer > System > Settings
F437	Branding colours and receipt header	Configuration	In screen: Institute profile (name, logo, address)
F438	Academic year settings	Configuration	In screen: Institute profile (name, logo, address)
F439	Working days and hours	Configuration	In screen: Institute profile (name, logo, address)
F440	Language and locale	Configuration	In screen: Institute profile (name, logo, address)
F441	Currency and tax settings	Configuration	In screen: Institute profile (name, logo, address)
F442	Notification defaults	Configuration	In screen: Institute profile (name, logo, address)
F443	Feature flags (Remote Config)	Configuration	In screen: Institute profile (name, logo, address)
F444	Theme: glass / liquid intensity	Configuration	In screen: Institute profile (name, logo, address)
F445	Reduce motion and accessibility	Configuration	In screen: Institute profile (name, logo, address)
F446	Storage and cache settings	Configuration	In screen: Institute profile (name, logo, address)
F447	App version and update prompt	List / filter	In screen: Institute profile (name, logo, address)
F448	Legal pages (privacy / terms)	Screen	Drawer > System > Settings
F449	About and diagnostics	Screen	Drawer > System > Settings
F450	Admin profile edit	Action	In screen: About and diagnostics

Appendix B. The 50 Libraries

#	Library	Area	Purpose
1	Jetpack Compose BOM + UI	UI	Declarative UI toolkit; BOM pins compatible Compose versions
2	Compose Material 3	UI	Base components and theming tokens
3	Compose Foundation + Animation	UI	Gestures, lazy lists, shared-element and transition animation
4	Navigation Compose	UI	Type-safe routes, drawer + bottom-bar destinations, deep links
5	Lifecycle ViewModel + runtime-compose	UI	State holders; lifecycle-aware StateFlow collection
6	Haze (blur / glassmorphism)	UI effects	Backdrop blur for glass panels with API-level fallbacks
7	Backdrop / liquid-glass Compose library	UI effects	Lens and refraction style liquid surfaces (verify maintained version)
8	AGSL RuntimeShader (platform, API 33+)	UI effects	Custom shaders for liquid ripples and gradients; fallback below API 33
9	Filament / SceneView	3D	Real-time 3D models and hero scenes inside Compose
10	Lottie Compose	Motion	Vector animations for empty and success states
11	Coil 3	Images	Async image loading and caching with Compose
12	Vico	Charts	Compose-native dashboard charts
13	Media3 ExoPlayer	Media	Recorded lecture playback with caching
14	CameraX	Media	Photo, document and QR capture
15	ML Kit Barcode Scanning	Media	QR attendance and ID card scanning
16	Splash Screen API	UI	Cold-start splash on all API levels
17	Hilt (Dagger)	DI	Compile-time dependency injection per module
18	Kotlin Coroutines + Flow	Core	Structured concurrency and reactive streams
19	kotlinx.serialization	Core	Typed JSON for callable-function payloads
20	Room (+ KSP compiler)	Local data	Offline cache of hot tables
21	DataStore (Preferences / Proto)	Local data	Settings and small state, replaces SharedPreferences
22	Paging 3	Data	Cursor-style paging of large lists
23	WorkManager	Background	Durable offline queues and scheduled sync
24	Firebase BoM	Firebase	Pins compatible Firebase SDK versions
25	Firebase Authentication	Firebase	Identity, ID tokens, custom claims
26	Cloud Firestore	Firebase	Primary database with listeners and offline cache
27	Cloud Storage for Firebase	Firebase	Photos, documents, certificates, material
28	Cloud Functions (callable client)	Firebase	Typed calls to server-only logic
29	Firebase Cloud Messaging	Firebase	Push notifications
30	Firebase App Check (Play Integrity provider)	Security	Attests that calls come from the genuine app
31	Firebase Crashlytics	Quality	Crash and non-fatal reporting
32	Firebase Performance Monitoring	Quality	Startup, network and screen traces
33	Firebase Remote Config	Ops	Feature flags and kill switches
34	Firebase Analytics	Ops	Usage events (no personal data)
35	Agora RTC SDK (Android)	Live	Voice, video and screen share
36	Play Integrity API	Security	Device and app integrity verdicts
37	Google Tink (AEAD)	Security	Authenticated encryption with Android Keystore keys
38	AndroidX Biometric	Security	Fingerprint / face unlock
39	SQLCipher for Android	Security	Encrypted Room database where on-device PII exists
40	OkHttp (+ CertificatePinner)	Network	Pinned client for any first-party HTTPS endpoint
41	RootBeer (signal only)	Security	Extra root signal, combined with Play Integrity
42	R8 / ProGuard rules	Security	Shrinking, obfuscation, dead-code removal
43	Timber	Quality	Logging with release tree that drops sensitive data
44	LeakCanary (debug only)	Quality	Memory-leak detection in debug builds
45	JUnit 4/5 + Truth	Test	Unit assertions

#	Library	Area	Purpose
46	MockK + Turbine	Test	Mocking and Flow testing
47	Compose UI Test + Espresso	Test	Instrumented UI tests
48	Macrobenchmark + Baseline Profiles	Performance	Startup and scroll performance with AOT profiles
49	Detekt + ktlint	Quality	Static analysis and formatting gates in CI
50	Android Lint + dependency-check	Quality	Lint and vulnerable-dependency scanning

Add a library to a module only when a feature needs it. Versions are declared in the version catalog and verified at implementation time.

Appendix C. The 50 Security Controls

Identity & access

#	Control	Implementation	Enforced at
C01	Server-side role claims	Roles (admin, teacher, student, parent) set only by Admin SDK as custom claims	Cloud Function + Rules
C02	Deny-by-default Firestore Rules	Every path closed unless a rule opens it	Firestore Rules
C03	Mandatory MFA for admin	TOTP second factor required before any admin screen loads	Auth + app gate
C04	Re-authentication for sensitive actions	Password change, refund approval, data export require fresh sign-in	App + Function
C05	Per-record ownership checks	Student/parent read only documents linked to their UID	Firestore Rules
C06	Teacher batch scoping	Teacher claims carry batch IDs; rules compare against document batch	Firestore Rules
C07	Session revocation	Refresh tokens revoked on role change, lock or password reset	Cloud Function
C08	Login lockout and throttling	Progressive delay after repeated failures, alert to admin	Auth + Function
C09	Four-eyes approval	Refunds, bulk deletes and role changes need a second approver	Function + audit
C10	Admin bootstrap outside the app	First admin created by a one-time script run by the owner; no default password anywhere	Ops procedure

Data protection

#	Control	Implementation	Enforced at
C11	Schema and type validation in Rules	Field types, lengths and enums validated on write	Firestore Rules
C12	Immutable audit log	Append-only audit collection, no client update or delete	Rules + Function
C13	Encryption in transit	TLS 1.2+ only; cleartext disabled	Manifest + network config
C14	Encryption at rest	Google-managed encryption for Firestore and Storage; CMEK optional	Google Cloud
C15	Data minimisation	Collect only fields a function needs; no sensitive data in notifications	Design review
C16	PII-safe logging	Release logging tree drops names, phones and tokens	Timber release tree
C17	Soft delete with retention	Archive first, purge by policy job	Function
C18	Storage rules by MIME and size	Only allowed types, size caps, path ownership	Storage Rules
C19	Field-level privacy for parents	Parents see only fee and result fields exposed to them	Rules + view docs
C20	Encrypted backups	Scheduled export to a locked bucket with restricted IAM	Cloud Scheduler + IAM

Network & transport

#	Control	Implementation	Enforced at
C21	Cleartext traffic disabled	usesCleartextTraffic=false and a network security config	Manifest
C22	Certificate pinning for first-party endpoints	OkHttp CertificatePinner with backup pin and rotation plan	App
C23	App Check enforced	Firestore, Storage and Functions reject unattested clients	Firebase
C24	Callable functions require auth	Every callable verifies auth.uid and claims	Cloud Function
C25	Rate limiting per user and per IP	Quota counters on AI, OTP and login-alias calls	Cloud Function
C26	Request size and schema limits	Reject oversized or malformed payloads early	Cloud Function
C27	CORS and webhook hardening	Payment webhooks verified by signature and idempotency key	Cloud Function
C28	Restricted API keys	Android key limited to package name + SHA-256 fingerprint	Google Cloud console

Client hardening

#	Control	Implementation	Enforced at
C29	R8 minification and obfuscation	Release builds shrunk and obfuscated; mapping uploaded privately	Gradle

#	Control	Implementation	Enforced at
C30	Play Integrity verdict check	Server checks device, app and licensing verdict for sensitive actions	Function
C31	Root / emulator signals	Signals feed risk score; high-risk blocks admin sign-in	App + Function
C32	Debugger and tamper checks	Detect debuggable flag, bad signature, hooking frameworks	App
C33	FLAG_SECURE on sensitive screens	Blocks screenshots and screen recording on fees and IDs	App
C34	Biometric gate on resume	Admin app locks after background timeout	App
C35	Keystore-backed secrets at rest	Tink AEAD with hardware-backed key; no plaintext prefs	App
C36	SQLCipher for any local PII	Encrypted database passphrase held in Keystore	App
C37	Exported components minimised	Only the launcher activity exported; intent filters verified	Manifest
C38	Backup and screenshot policy	allowBackup=false or explicit data-extraction rules	Manifest

Secrets & backend

#	Control	Implementation	Enforced at
C39	No provider keys in APK	Gemini, OpenAI and Agora certificate never enter the client	Architecture
C40	Google Secret Manager	Provider keys stored as secrets, bound to functions only	Cloud Functions
C41	Least-privilege service accounts	Each function runs with a narrow IAM role	IAM
C42	Key rotation runbook	Scheduled rotation with zero-downtime secret versions	Ops
C43	Provider-side spend limits	Budgets and alerts on Gemini, OpenAI and Agora consoles	Provider consoles
C44	Short-lived Agora tokens	Server mints per-user, per-channel tokens with minutes of lifetime	Cloud Function

Build & operations

#	Control	Implementation	Enforced at
C45	Secret scanning in CI	Gitleaks on every push and pull request	GitHub Actions
C46	Dependency and licence review	Dependabot plus dependency-review action	GitHub
C47	Signed release from protected secrets	Keystore only in GitHub encrypted secrets, decoded at build time	GitHub Actions
C48	Branch protection	Required reviews and green checks before merge to main	GitHub
C49	Release gate	Manual approval via protected environment before distribution	GitHub Environments
C50	Monitoring and alerting	Crashlytics, error-rate and budget alerts to the owner	Firebase + GCP

Appendix D. Video Walkthrough Index

The companion video (waqar_android_backend_architecture_walkthrough.mp4, about five minutes) contains 13 slides. Frames are embedded in the chapters where each topic is discussed.

Slide	Topic	Where used in this document
1	Whole system at a glance	Chapter 4, Figure 4.1
2	What happens inside the app	Chapter 5, Figure 5.1
3	Login and the four roles	Chapter 6, Figure 6.1
4	Firebase SDK versus server-only work	Chapter 7, Figure 7.1
5	Real data flow of a student save	Chapter 9, Figure 9.1
6	Secure Gemini and OpenAI connection	Chapter 13, Figure 13.1
7	Agora voice and video call	Chapter 13, Figure 13.2
8	Where the fifty libraries fit	Chapter 15, Figure 15.1
9	Which Kotlin files do what	Chapter 15, Figure 15.2
10	Where fifty security controls apply	Chapter 14, Figure 14.3
11	Offline sync and 100,000-record performance	Chapter 9, Figure 9.2
12	GitHub Actions to native APK	Chapter 16, Figure 16.2
13	Final verdict	Chapter 19, Figure 19.1

Reference diagram index

Diagram	Used in
Google Cloud grouping (authentication, application, data, AI)	Figure 4.3
Component view with secure backend	Figure 4.2
MVVM Clean Architecture	Figure 5.2
Clean Architecture and SOLID	Figure 5.3
Dagger components and subcomponents	Figure 5.4
Firebase core components	Figure 7.2
Client SDKs to Firebase	Figure 7.3
Mobile client with BigQuery analytics	Figure 7.4
Firestore physical layout	Figure 8.1
Layered security approach in Android	Figure 14.1
L1 / L2 / R security levels	Figure 14.2
CI flow (ASCII)	Figure 16.1
GitHub Actions development to production	Figure 16.3
Android CI/CD pipeline	Figure 16.4